

ShipZen: The Ultimate Deep-Dive Study Guide

This document is a comprehensive, multi-page study guide designed to prepare you for Staff-Level architecture reviews and technical interviews. It covers every component, file, and line of code in the ShipZen project.

1. Executive Summary & Architecture

ShipZen is an event-driven Platform-as-a-Service (PaaS) built on AWS EKS. It abstracts Kubernetes complexity by allowing users to deploy containerized applications from GitHub with zero configuration.

Key Technologies & Justifications

- **FastAPI:** Chosen for async performance and auto-generated OpenAPI documentation.
- **Go (Kaniko Worker):** Chosen for low memory footprint and concurrency. Better than Tekton for simple use cases.
- **Karpenter:** Selected over Cluster Autoscaler for faster node provisioning directly from EC2 Fleet APIs.
- **Envoy Gateway:** Chosen over NGINX for native Gateway API support and dynamic xDS routing without reloads.

2. Infrastructure & Kubernetes Deep Dive

VPC & Networking

The cluster resides in a private VPC. AWS ALBs exist in the public subnets, forwarding traffic to Envoy pods in the private subnets. Tenant workloads run on dedicated Spot instances managed by Karpenter.

3. Controller & Operator Deep Dive

File: `controller/main.py`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\controller` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
import os
import time
import logging
import yaml
import psycpg2
from psycpg2.extras import DictCursor
from jinja2 import Environment, FileSystemLoader
from kubernetes import client, config as k8s_config
from kubernetes.client.rest import ApiException
from kubernetes.utils import create_from_yaml
import boto3
import redis
import json
from models import ProjectStatus, ProjectSchema
from metrics import (
    shipzen_drift_total,
    shipzen_reconciliation_duration_seconds,
    shipzen_deployment_success_total,
```

```

        start_metrics_server
    )

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - ',
                    logger = logging.getLogger('controller'))

try:
    k8s_config.load_incluster_config()
except k8s_config.ConfigException:
    k8s_config.load_kube_config()

k8s_client = client.ApiClient()
k8s_core_api = client.CoreV1Api()
k8s_apps_api = client.AppsV1Api()
k8s_custom_api = client.CustomObjectsApi()

# Fix #20: raise if env var is missing rather than silently falling back to
# a hardcoded plaintext credential that will never work in-cluster.
DATABASE_URL = os.environ.get("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL environment variable is not set")

RECONCILIATION_INTERVAL = int(os.getenv("RECONCILIATION_INTERVAL", "15"))

# ECR registry hostname – used when rendering the tenant namespace template
# so each tenant namespace gets an ECR pull secret via ESO.
# Format: 123456789012.dkr.ecr.us-east-1.amazonaws.com
ECR_REGISTRY = os.getenv("ECR_REGISTRY", "")

jinja_env = Environment(loader=FileSystemLoader("templates"))

def ensure_ecr_repository(project_id: str):
    try:
        ecr = boto3.client('ecr', region_name=os.getenv("AWS_REGION", "us-east-1"))
        repo_name = f"shipzen-builds/{project_id}"
        try:
            ecr.describe_repositories(repositoryNames=[repo_name])
        except ecr.exceptions.RepositoryNotFoundException:
            logger.info(f"Creating ECR repository {repo_name}")
            ecr.create_repository(
                repositoryName=repo_name,
                imageScanningConfiguration={'scanOnPush': True},
                imageTagMutability='IMMUTABLE'
            )
    )

```

```

except Exception as e:
    logger.error(f"Failed to ensure ECR repository exists: {e}")

def get_db_connection():
    conn = psycopg2.connect(DATABASE_URL)
    # Fix #25: autocommit = False so PROVISIONING → READY transitions are
    # atomic. A crash after apply_manifests() but before the UPDATE will
    # roll back, leaving the project in PROVISIONING for the next loop.
    conn.autocommit = False
    return conn

def _wait_for_schema(max_attempts: int = 30, delay: int = 10):
    """
    Block until the schema bootstrap Job has run and the projects table exists.
    On a fresh cluster the Job runs as an ArgoCD PostSync hook – this can take
    30-60s after the controller pod starts. Without this guard the controller
    would crash-loop with 'relation "projects" does not exist'.
    """
    for attempt in range(1, max_attempts + 1):
        conn = None
        try:
            conn = get_db_connection()
            with conn.cursor() as cur:
                cur.execute("SELECT 1 FROM projects LIMIT 1;")
            conn.close()
            logger.info("Database schema is ready.")
            return
        except psycopg2.OperationalError as e:
            logger.warning(f"DB not reachable yet (attempt {attempt}/{max_attempts}): {e}")
        except psycopg2.errors.UndefinedTable:
            logger.warning(f"Schema not ready yet (attempt {attempt}/{max_attempts}), wait")
        except Exception as e:
            logger.warning(f"Unexpected DB error (attempt {attempt}/{max_attempts}): {e}")
        finally:
            if conn is not None:
                try:
                    conn.close()
                except Exception:
                    pass
            time.sleep(delay)
    raise RuntimeError(f"Database schema not ready after {max_attempts * delay}s – aborting")

```

```

def apply_manifests(manifest_str: str):
    """
    Parses the multi-document YAML and applies each document via the
    kubernetes Python client. If the resource already exists, it is patched.
    """
    logger.info("Applying K8s manifests via Python client...")
    docs = list(yaml.safe_load_all(manifest_str))
    for doc in docs:
        if doc is None:
            continue
        try:
            create_from_yaml(k8s_client, yaml_objects=[doc], verbose=False)
            logger.info(f"Applied: {doc.get('kind', 'unknown')} / {doc.get('metadata', {})}")
        except ApiException as e:
            if e.status == 409:
                kind = doc.get("kind")
                name = doc.get("metadata", {}).get("name")
                namespace = doc.get("metadata", {}).get("namespace", "default")
                try:
                    if kind == "Deployment":
                        k8s_apps_api.patch_namespaced_deployment(name, namespace, doc)
                    elif kind == "Service":
                        k8s_core_api.patch_namespaced_service(name, namespace, doc)
                    elif kind == "HTTPRoute":
                        k8s_custom_api.patch_namespaced_custom_object(
                            "gateway.networking.k8s.io", "v1", namespace, "httproutes", name, doc)
                    elif kind == "ExternalSecret":
                        k8s_custom_api.patch_namespaced_custom_object(
                            "external-secrets.io", "v1", namespace, "externalsecrets", name, doc)
                    elif kind == "PodDisruptionBudget":
                        client.PolicyV1Api().patch_namespaced_pod_disruption_budget(name, namespace, doc)
                    elif kind == "NetworkPolicy":
                        client.NetworkingV1Api().patch_namespaced_network_policy(name, namespace, doc)
                    elif kind == "ResourceQuota":
                        k8s_core_api.patch_namespaced_resource_quota(name, namespace, doc)
                    elif kind == "LimitRange":
                        k8s_core_api.patch_namespaced_limit_range(name, namespace, doc)
                    elif kind == "Role":
                        client.RbacAuthorizationV1Api().patch_namespaced_role(name, namespace, doc)
                    elif kind == "RoleBinding":
                        client.RbacAuthorizationV1Api().patch_namespaced_role_binding(name, namespace, doc)
                except ApiException as e:
                    logger.warning(f"Patching not implemented for {kind}")
                    continue
                logger.info(f"Patched: {kind} / {name}")

```

```

        except Exception as patch_e:
            logger.error(f"Failed to patch {kind} {name}: {patch_e}")
        else:
            logger.warning(f"apply_manifests API error for {doc.get('kind')}: {e}")
    except Exception as e:
        if hasattr(e, "status") and e.status == 409:
            logger.warning(f"Caught 409 inside generic exception for {doc.get('kind')}")
            logger.warning(f"apply_manifests error for {doc.get('kind')}: {e}")

def check_namespace_exists(namespace: str) -> bool:
    try:
        k8s_core_api.read_namespace(name=namespace)
        return True
    except ApiException as e:
        if e.status == 404:
            return False
        raise

def delete_namespace(namespace: str):
    try:
        k8s_core_api.delete_namespace(name=namespace)
    except ApiException as e:
        if e.status != 404:
            raise

@shipzen_reconciliation_duration_seconds.time()
def reconcile():
    logger.info("Starting reconciliation loop...")
    conn = get_db_connection()
    try:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute("SELECT * FROM projects;")
            projects = cur.fetchall()

            for row in projects:
                project_data = dict(row)
                project_data["_id"] = project_data.pop("id")

                try:
                    project = ProjectSchema(**project_data)

                    with conn.cursor(cursor_factory=DictCursor) as project_cur:

```

```

if project.status == ProjectStatus.PROVISIONING:
    logger.info(f"Provisioning project: {project.name} ({project.
template = jinja_env.get_template("tenant.yaml.j2")
manifests = template.render(
    namespace=project.namespace,
    project_id=project.id,
    ecr_registry=ECR_REGISTRY,
)
# Fix #1: actually applies manifests now.
apply_manifests(manifests)

# Fix #12: only mark READY after verifying the namespace
# was actually created. This breaks the race where the DB
# is marked READY before K8s has processed the request.
if check_namespace_exists(project.namespace):
    ensure_ecr_repository(project.id)
    project_cur.execute(
        "UPDATE projects SET status = %s WHERE id = %s;",
        (ProjectStatus.READY.value, project.id)
    )
    conn.commit()
    logger.info(f"Project {project.name} provisioned and Read
else:
    # Namespace not visible yet; leave as PROVISIONING
    # and retry on the next reconcile tick.
    conn.rollback()
    logger.info(f"Namespace {project.namespace} not yet visib

elif project.status == ProjectStatus.TERMINATING:
    logger.info(f"Terminating project: {project.name} ({project.n
if check_namespace_exists(project.namespace):
    delete_namespace(project.namespace)
    logger.info(f"Namespace {project.namespace} deletion trig
    conn.commit()
else:
    project_cur.execute("DELETE FROM projects WHERE id = %s;"
    conn.commit()
    logger.info(f"Project {project.name} permanently cleaned

elif project.status == ProjectStatus.READY:
    if not check_namespace_exists(project.namespace):
        shipzen_drift_total.inc()
        logger.warning(f"Drift detected! Namespace {project.names
    project_cur.execute(
        "UPDATE projects SET status = %s WHERE id = %s;",

```



```

        (ProjectStatus.PROVISIONING.value, project.id)
    )
    conn.commit()
else:
    reconcile_deployments(conn, project_cur, project)

except Exception as e:
    logger.error(f"Error reconciling project {row['id']}: {e}")
    try:
        conn.rollback()
        with get_db_connection() as err_conn:
            with err_conn.cursor() as err_cur:
                err_cur.execute(
                    "UPDATE projects SET status = %s WHERE id = %s;",
                    (ProjectStatus.FAILED.value, row['id'])
                )
            err_conn.commit()
    except Exception as rb_err:
        logger.error(f"Failed to set FAILED state: {rb_err}")

finally:
    conn.close()

def reconcile_deployments(conn, cur, project):
    """Reconciles Deployments, Services, and HTTPRoutes for a ready project namespace."""
    cur.execute("SELECT * FROM deployments WHERE project_id = %s;", (project.id,))
    db_deployments = {str(row['deployment_id']): dict(row) for row in cur.fetchall()}

    try:
        k8s_deps = k8s_apps_api.list_namespaced_deployment(namespace=project.namespace)
        k8s_dep_names = {d.metadata.name: d for d in k8s_deps.items}

        k8s_svcs = k8s_core_api.list_namespaced_service(namespace=project.namespace)
        k8s_svc_names = {s.metadata.name for s in k8s_svcs.items}

        try:
            k8s_routes = k8s_custom_api.list_namespaced_custom_object(
                "gateway.networking.k8s.io", "v1", namespace=project.namespace, plural="httproutes"
            )
            k8s_route_names = {r['metadata']['name'] for r in k8s_routes.get('items', [])}
        except ApiException:
            k8s_route_names = set()

    # 1. Missing or Drifted Deployments

```

```

for d_id, db_dep in db_deployments.items():
    if db_dep['state'] in ['Running', 'Verifying', 'Deploying']:
        missing_resources = (
            d_id not in k8s_dep_names or
            f"{d_id}-svc" not in k8s_svc_names or
            f"{d_id}-route" not in k8s_route_names
        )
        if missing_resources:
            shipzen_drift_total.inc()
            logger.warning(f"Drift: Deployment {d_id} or its resources missing in
            template = jinja_env.get_template("app-deployment.yaml.j2")
            manifests = template.render(
                deployment_name=d_id,
                deployment_id=d_id,
                namespace=project.namespace,
                project_name=project.name,
                image_uri=db_dep.get('image_uri', 'nginx:latest'),
                port=db_dep.get('port', 8080),
                replicas=db_dep.get('replicas', 1),
                health_check_path=db_dep.get('health_check_path', '/')
            )
            apply_manifests(manifests)
        else:
            k8s_dep = k8s_dep_names[d_id]
            ready_replicas = k8s_dep.status.ready_replicas or 0
            if ready_replicas == 0 and db_dep['state'] == 'Running':
                shipzen_drift_total.inc()
                logger.warning(f"Drift: Deployment {d_id} is failing in K8s.")
                cur.execute(
                    "UPDATE deployments SET state = %s, last_error = %s WHERE dep
                    ('Failed', 'Kubernetes Deployment Failed/CrashLoopBackOff', d
                )
                conn.commit()
                try:
                    r = redis.Redis(host=os.getenv("REDIS_HOST", "redis-master.sh
                    r.publish(f"shipzen:status:{d_id}", json.dumps({"state": "Fai
                except Exception as pub_e:
                    logger.warning(f"Failed to publish to Redis: {pub_e}")
            elif ready_replicas > 0 and db_dep['state'] in ['Deploying', 'Verifi
                logger.info(f"Deployment {d_id} is now Running (Ready Replicas: {
                cur.execute(
                    "UPDATE deployments SET state = %s, last_error = NULL WHERE d
                    ('Running', d_id)
                )
                conn.commit()

```

```

        shipzen_deployment_success_total.inc()
    try:
        r = redis.Redis(host=os.getenv("REDIS_HOST", "redis-master.sh
        r.publish(f"shipzen:status:{d_id}", json.dumps({"state": "Run
    except Exception as pub_e:
        logger.warning(f"Failed to publish to Redis: {pub_e}")

# 2. Orphan Resources Cleanup
for k8s_name in k8s_dep_names.keys():
    if k8s_name not in db_deployments or db_deployments[k8s_name]['state'] not in
        shipzen_drift_total.inc()
    logger.warning(f"Drift: Orphan Deployment {k8s_name} found in K8s. Cleani
    k8s_apps_api.delete_namespaced_deployment(name=k8s_name, namespace=projec
    try:
        k8s_core_api.delete_namespaced_service(name=f"{k8s_name}-svc", namesp
        k8s_custom_api.delete_namespaced_custom_object(
            group="gateway.networking.k8s.io",
            version="v1",
            namespace=project.namespace,
            plural="httproutes",
            name=f"{k8s_name}-route"
        )
    except ApiException:
        pass

except Exception as e:
    logger.error(f"Error reconciling deployments for {project.name}: {e}")

def main():
    # Fix #2: metrics on 9090, not 8080 (defined in metrics.py default)
    start_metrics_server(port=9090)
    _wait_for_schema()
    while True:
        reconcile()
        time.sleep(RECONCILIATION_INTERVAL)

if __name__ == "__main__":
    main()

```

File: controller\metrics.py

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\controller subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
from prometheus_client import Counter, Summary, start_http_server

# Fix: was a Gauge – rate() on a Gauge always returns 0 and the drift alert
# was silently broken. Counter is correct for a monotonically increasing event total.
shipzen_drift_total = Counter(
    'shipzen_drift_total',
    'Total number of detected drift incidents (missing or orphaned resources)'
)

shipzen_reconciliation_duration_seconds = Summary(
    'shipzen_reconciliation_duration_seconds',
    'Time spent in the reconciliation loop'
)

# Fix #19: Counter resets on restart, but we will use PromQL increase()
# or rate() to measure success rate over time instead of relying on absolute value.
shipzen_deployment_success_total = Counter(
    'shipzen_deployment_success_total',
    'Total number of successful deployments transitioned to Running'
)

def start_metrics_server(port: int = 9090):
    # Fix: was 8080, which collides with the controller's own app port.
    # Metrics now bind on 9090.
    start_http_server(port)
```

File: controller\models.py

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\controller subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
from typing import Optional
from datetime import datetime
from pydantic import BaseModel, Field, ConfigDict, field_serializer
from enum import Enum

class ProjectStatus(str, Enum):
    PROVISIONING = "Provisioning"
    READY = "Ready"
    TERMINATING = "Terminating"
    FAILED = "Failed"

class ProjectSchema(BaseModel):
    model_config = ConfigDict(populate_by_name=True)
    id: str = Field(alias="_id")
    name: str
    namespace: str
    status: ProjectStatus = Field(default=ProjectStatus.PROVISIONING)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    deleted_at: Optional[datetime] = None

    @field_serializer('created_at', 'deleted_at')
    def serialize_dt(self, dt: datetime, _info):
        return dt.isoformat() if dt else None
```

File: controller\templates\app-deployment.yaml.j2

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\controller subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```

apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: {{ deployment_name }}-secrets
  namespace: {{ namespace }}
  labels:
    shipzen.jeneeldumasia.codes/project: "{{ project_name }}"
spec:
  refreshInterval: "1h"
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: {{ deployment_name }}-secrets-sync
    creationPolicy: Owner
  dataFrom:
    - extract:
        # Assumes secrets in AWS SM are stored under a path scoped to the tenant/project
        key: shipzen/{{ project_name }}/
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ deployment_name }}
  namespace: {{ namespace }}
  labels:
    app: {{ deployment_name }}
    shipzen.jeneeldumasia.codes/project: "{{ project_name }}"
spec:
  replicas: {{ replicas | default(1) }}
  selector:

```

```

matchLabels:
  app: {{ deployment_name }}
template:
  metadata:
    labels:
      app: {{ deployment_name }}
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "{{ port | default(8080) }}"
      prometheus.io/path: "/metrics"
  spec:
    # Fix #5.4: toleration to schedule on the dedicated tenant NodePool.
    # Tenant nodes are tainted shipzen.jeneeldumasia.codes/dedicated=tenant:NoSchedule
    # so builder workloads cannot mix with tenant app pods.
    tolerations:
      - key: shipzen.jeneeldumasia.codes/dedicated
        operator: Equal
        value: tenant
        effect: NoSchedule
    # Fix #8.8: imagePullSecret so tenant pods can pull built images from ECR.
    # The ecr-pull-secret is created per-namespace by the controller when
    # provisioning the tenant namespace (via a Kubernetes Job that runs
    # `aws ecr get-login-password` and creates the secret).
    securityContext:
      runAsNonRoot: true
      seccompProfile:
        type: RuntimeDefault
    imagePullSecrets:
      - name: ecr-pull-secret
    containers:
      - name: app
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
              - ALL
        image: {{ image_uri }}
        env:
          - name: PORT
            value: "{{ port | default(8080) }}"
        envFrom:
          - secretRef:
              name: {{ deployment_name }}-secrets-sync
              optional: true
        ports:

```

```

      - name: http
        containerPort: {{ port | default(8080) }}
    readinessProbe:
      httpGet:
        path: "{{ health_check_path | default('/') }}"
        port: {{ port | default(8080) }}
      initialDelaySeconds: 5
      periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: {{ deployment_name }}-svc
  namespace: {{ namespace }}
  labels:
    app: {{ deployment_name }}
    shipzen.jeneeldumasia.codes/project: "{{ project_name }}"
spec:
  selector:
    app: {{ deployment_name }}
  ports:
    - protocol: TCP
      port: 80
      targetPort: {{ port | default(8080) }}
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: {{ deployment_name }}-route
  namespace: {{ namespace }}
  labels:
    shipzen.jeneeldumasia.codes/project: "{{ project_name }}"
spec:
  parentRefs:
    - name: shipzen-gateway
      namespace: shipzen-system
  hostnames:
    - "{{ deployment_id[:8] }}-{{ project_name }}-shipzen.jeneeldumasia.codes"
  rules:
    - backendRefs:
        - name: {{ deployment_name }}-svc
          port: 80
---
apiVersion: policy/v1
kind: PodDisruptionBudget

```



```
metadata:
  name: {{ deployment_name }}-pdb
  namespace: {{ namespace }}
  labels:
    shipzen.jeneeldumasia.codes/project: "{{ project_name }}"
spec:
  # Fix #19: minAvailable: 1 on a single-replica deployment makes the pod
  # permanently un-evictable, blocking Karpenter consolidation and node drains.
  # Use maxUnavailable: 1 instead – this allows voluntary evictions while
  # still protecting multi-replica deployments from losing all pods at once.
  maxUnavailable: 1
  selector:
    matchLabels:
      app: {{ deployment_name }}
```

File: controller\templates\tenant.yaml.j2

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\controller subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```

apiVersion: v1
kind: Namespace
metadata:
  name: {{ namespace }}
  labels:
    shipzen.jeneeldumasia.codes/project-id: "{{ project_id }}"
    shipzen.jeneeldumasia.codes/tenant: "true"
    # Basic pod security isolation
    pod-security.kubernetes.io/enforce: restricted
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: default
  namespace: {{ namespace }}
automountServiceAccountToken: false
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-quota
  namespace: {{ namespace }}
spec:
  hard:
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
    pods: "20"
    services: "10"
---
```

```

apiVersion: v1
kind: LimitRange
metadata:
  name: tenant-limit-range
  namespace: {{ namespace }}
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      type: Container
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: tenant-isolation
  namespace: {{ namespace }}
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
  # Default deny all ingress unless specifically allowed via Gateway
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
                kubernetes.io/metadata.name: shipzen-system # Allow traffic from Envoy Gate
  egress:
    # Explicitly allow DNS
    - ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    # Block internal Kubernetes API & Metadata endpoint
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
            except:
                - 10.0.0.0/8 # Typical Cluster VPC/Pod CIDR
                - 172.16.0.0/12

```

```

- 192.168.0.0/16
- 100.64.0.0/10 # Carrier-grade NAT / EKS VPC CNI secondary ranges
- 169.254.169.254/32 # Block IMDS explicitly as requested

---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: tenant-runner
  namespace: {{ namespace }}
rules:
  # Task 17 / fix #4.7: removed "secrets" from resources.
  # Apps receive their secrets as environment variables injected by ESO –
  # they have no legitimate reason to call the Kubernetes secrets API directly.
  # Granting list/get on secrets allowed any tenant pod to enumerate all
  # credentials in the namespace, including other deployments' synced secrets.
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: tenant-runner-binding
  namespace: {{ namespace }}
subjects:
  - kind: ServiceAccount
    name: default
    namespace: {{ namespace }}
roleRef:
  kind: Role
  name: tenant-runner
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: generators.external-secrets.io/v1alpha1
kind: ECRAuthorizationToken
metadata:
  name: ecr-token-generator
  namespace: {{ namespace }}
spec:
  region: us-east-1
  # Role isn't strictly necessary if the ESO pod IRSA role has ecr:GetAuthorizationToken
---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
```

```

    name: ecr-pull-secret
    namespace: {{ namespace }}
spec:
  refreshInterval: "1h"
  target:
    name: ecr-pull-secret
    template:
      type: kubernetes.io/dockerconfigjson
      data:
        .dockerconfigjson: |
          {"auths":{"{{ ecr_registry }}":{"username":"{{ .username }}" }, "password"
dataFrom:
  - sourceRef:
      generatorRef:
        apiVersion: generators.external-secrets.io/v1alpha1
        kind: ECRAuthorizationToken
        name: ecr-token-generator
---
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: tenant-pod-monitor
  namespace: {{ namespace }}
  labels:
    release: kube-prometheus-stack
spec:
  selector:
    matchExpressions:
      - key: app
        operator: Exists
  podMetricsEndpoints:
    - port: http
      path: /metrics

```

4. Build Worker Deep Dive

5. API & Backend Deep Dive

File: `api\analyzer.py`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\api` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
import os
from pathlib import Path
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class DetectedService:
    name: str
    path: str # Relative to repo root
    type: str # 'node', 'python', 'static', 'go', etc.
    framework: Optional[str] = None
    entrypoint: Optional[str] = None

class RepoAnalyzer:
    def __init__(self, repo_path: str | Path, repo_name: str | None = None):
        self.repo_path = Path(repo_path)
        # Use provided name, or fall back to the directory name
        self._repo_name = repo_name or self.repo_path.name

    def analyze(self) -> List[DetectedService]:
        services = []
        skip_dirs = {'node_modules', 'venv', '.venv', '__pycache__', 'dist', 'build'}

        base_path = self.repo_path.resolve()
        if not base_path.exists() or not base_path.is_dir():
```

```

        return services

def walk_path(current_path: Path):
    if current_path != base_path:
        if current_path.name in skip_dirs or current_path.name.startswith('.'):
            return

    rel_path = ""
    if current_path != base_path:
        rel_path = current_path.relative_to(base_path).as_posix()

    try:
        files = {p.name for p in current_path.iterdir() if p.is_file()}
    except Exception:
        return

    if 'package.json' in files:
        services.append(self._analyze_node(str(current_path), rel_path))
    elif any(f in files for f in ['requirements.txt', 'pyproject.toml', 'manage.p
        services.append(self._analyze_python(str(current_path), rel_path))
    elif 'composer.json' in files or 'index.php' in files:
        services.append(DetectedService(name=current_path.name or self._repo_name
    elif 'go.mod' in files:
        services.append(DetectedService(name=current_path.name or self._repo_name
    elif 'Cargo.toml' in files:
        services.append(DetectedService(name=current_path.name or self._repo_name
    elif 'pom.xml' in files or 'build.gradle' in files:
        services.append(DetectedService(name=current_path.name or self._repo_name
    elif 'Gemfile' in files:
        services.append(DetectedService(name=current_path.name or self._repo_name
    elif 'index.html' in files and not any(s.path == rel_path for s in services):
        basename = current_path.name.lower()
        if rel_path == "" or basename in {'public', 'dist', 'build', 'www', 'html
            services.append(DetectedService(
                name=current_path.name or self._repo_name,
                path=rel_path,
                type="static"
            ))

    try:
        for d in current_path.iterdir():
            if d.is_dir():
                walk_path(d)
    except Exception:
        pass

```

```

walk_path(base_path)
return services

def _analyze_node(self, full_path: str, rel_path: str) -> DetectedService:
    # If at repo root, use repo name; otherwise use directory name
    name = (self._repo_name if not rel_path else os.path.basename(full_path)) or "app"
    framework = None

    # Simple framework detection
    pkg_json_path = Path(full_path) / "package.json"
    try:
        with open(pkg_json_path, 'r') as f:
            content = f.read()
            if '"next"' in content: framework = "nextjs"
            elif '"vite"' in content: framework = "vite"
            elif '"express"' in content: framework = "express"
    except:
        pass

    return DetectedService(
        name=name,
        path=rel_path,
        type="node",
        framework=framework
    )

def _analyze_python(self, full_path: str, rel_path: str) -> DetectedService:
    # If at repo root, use repo name; otherwise use directory name
    name = (self._repo_name if not rel_path else os.path.basename(full_path)) or "app"
    framework = None

    # Simple framework detection
    files = os.listdir(full_path)
    if 'manage.py' in files: framework = "django"

    req_path = Path(full_path) / "requirements.txt"
    if req_path.exists():
        try:
            content = req_path.read_text().lower()
            if "fastapi" in content: framework = "fastapi"
            elif "flask" in content: framework = "flask"
        except:
            pass

```



```
return DetectedService(  
    name=name,  
    path=rel_path,  
    type="python",  
    framework=framework  
)
```

File: api\audit.py

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\api subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
import json
import logging
from database import get_connection
from psycopg2.extras import DictCursor

logger = logging.getLogger(__name__)

# Fix #9.3: hard cap on audit log query size to prevent runaway queries
MAX_AUDIT_LIMIT = 500

def log_audit_event(
    project_id: str,
    user_id: str,
    action: str,
    resource_type: str,
    resource_id: str,
    details: dict,
):
    """
    Appends an event to the audit_logs table.
    This table is logically append-only.

    Fix #3.4: exceptions are caught and logged rather than propagated.
    Audit logging is a side effect – a DB failure here must not crash the
    originating API request.
    """
    try:
        with get_connection() as conn:
            with conn.cursor() as cur:
                cur.execute(
```

```

        """
        INSERT INTO audit_logs
            (project_id, user_id, action, resource_type, resource_id, details
VALUES (%s, %s, %s, %s, %s, %s)
        """,
        (project_id, user_id, action, resource_type, resource_id, json.dumps(
            )
        logger.info(f"Audit event logged: {action} on {resource_type} {resource_id} by {u
except Exception as e:
    # Log and swallow – never let audit logging break the caller
    logger.error(f"Failed to write audit event ({action} / {resource_id}): {e}")

def get_audit_logs(project_id: str = None, user_id: str = None, limit: int = 50):
    """
    Queries the append-only audit log by project or user.

    Fix #9.3: limit is capped at MAX_AUDIT_LIMIT (500) to prevent a caller
    passing limit=1000000 from issuing a runaway query.
    """
    limit = min(int(limit), MAX_AUDIT_LIMIT)

    query = "SELECT * FROM audit_logs WHERE 1=1"
    params = []

    if project_id:
        query += " AND project_id = %s"
        params.append(project_id)
    if user_id:
        query += " AND user_id = %s"
        params.append(user_id)

    query += " ORDER BY timestamp DESC LIMIT %s;"
    params.append(limit)

    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(query, tuple(params))
            return [dict(row) for row in cur.fetchall()]

```

File: api\auth.py

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\api subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
import os
import logging
from dataclasses import dataclass
from typing import Optional

import httpx
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from cachetools import TTLCache

logger = logging.getLogger(__name__)

GITHUB_ENABLED = os.getenv("GITHUB_ENABLED", "false").lower() == "true"

_bearer = HTTPBearer(auto_error=False)

# Cache GitHub tokens for 5 minutes to avoid rate limits
_token_cache = TTLCache(maxsize=1000, ttl=300)

@dataclass
class User:
    user_id: str
    is_admin: bool = False

def get_current_user_from_token(token: str) -> User:
    return get_current_user(HTTPAuthorizationCredentials(scheme="Bearer", credentials=token))

def get_current_user(
    credentials: Optional[HTTPAuthorizationCredentials] = Depends(_bearer),
) -> User:
    if not GITHUB_ENABLED:
```

```

        logger.warning("GITHUB_ENABLED not true – using stub user for local dev")
        from database import get_or_create_user
        db_user = get_or_create_user("local-dev-user", "admin@shipzen.local")
        return User(user_id=db_user["id"], is_admin=(db_user["role"] == "admin"))

if credentials is None:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Authorization header missing",
        headers={"WWW-Authenticate": "Bearer"},
    )

token = credentials.credentials

# Check cache
if token in _token_cache:
    user_info = _token_cache[token]
else:
    # Verify token with GitHub
    try:
        resp = httpx.get(
            "https://api.github.com/user",
            headers={"Authorization": f"Bearer {token}", "Accept": "application/vnd.g"},
            timeout=5
        )
        if resp.status_code != 200:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Invalid GitHub token",
            )

        gh_user = resp.json()
        # Fetch emails because primary email might be private
        email_resp = httpx.get(
            "https://api.github.com/user/emails",
            headers={"Authorization": f"Bearer {token}", "Accept": "application/vnd.g"},
            timeout=5
        )
        email = None
        if email_resp.status_code == 200:
            for e in email_resp.json():
                if e.get("primary"):
                    email = e.get("email")
                    break

```

```
        user_info = {
            "id": str(gh_user["id"]),
            "login": gh_user["login"],
            "email": email or gh_user.get("email")
        }
        _token_cache[token] = user_info
    except httpx.RequestError as e:
        logger.error(f"GitHub API request failed: {e}")
        raise HTTPException(status_code=503, detail="Auth service unavailable")

from database import get_or_create_user
db_user = get_or_create_user(user_info["id"], user_info["email"])
return User(user_id=user_info["id"], is_admin=(db_user["role"] == "admin"))
```

File: api\database.py

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\api subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
import os
import psycopg2
from psycopg2.extras import DictCursor
import logging

logger = logging.getLogger(__name__)

# Fix #20: raise immediately on missing env var rather than silently falling
# back to postgres:postgres which will never connect inside the cluster.
DATABASE_URL = os.environ.get("DATABASE_URL", "")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL environment variable is not set")

from psycopg2.pool import ThreadedConnectionPool

db_pool = ThreadedConnectionPool(1, 20, dsn=DATABASE_URL)

class PooledConnectionWrapper:
    def __init__(self, conn, pool):
        self._conn = conn
        self._pool = pool

    def __enter__(self):
        self._conn.__enter__()
        return self._conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        try:
            self._conn.__exit__(exc_type, exc_val, exc_tb)
        finally:
```

```

        self._pool.putconn(self._conn)
        self._conn = None # Prevent double put

def cursor(self, *args, **kwargs):
    return self._conn.cursor(*args, **kwargs)

def commit(self):
    self._conn.commit()

def rollback(self):
    self._conn.rollback()

def close(self):
    if self._conn:
        self._pool.putconn(self._conn)
        self._conn = None

def get_connection():
    conn = db_pool.getconn()
    return PooledConnectionWrapper(conn, db_pool)

def get_deployments_paginated(project_id: str, limit: int = 20, cursor_updated_at: str =
    """
    Keyset pagination (cursor-based) – avoids OFFSET degradation on large tables.
    """
    query = """
        SELECT deployment_id, state, updated_at
        FROM deployments
        WHERE project_id = %s
    """
    params = [project_id]

    if cursor_updated_at:
        query += " AND updated_at < %s"
        params.append(cursor_updated_at)

    query += " ORDER BY updated_at DESC LIMIT %s;"
    params.append(limit)

    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(query, tuple(params))
            return [dict(row) for row in cur.fetchall()]

```



```

def get_or_create_user(user_id: str, email: str = None) -> dict:
    """Gets a user by ID, or creates them. The first user created gets the 'admin' role."""
    conn = get_connection()
    try:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute("SELECT id, role FROM users WHERE id = %s", (user_id,))
            user = cur.fetchone()
            if user:
                return dict(user)

            cur.execute("SELECT COUNT(*) FROM users")
            count = cur.fetchone()[0]
            role = 'admin' if count == 0 else 'user'

            cur.execute(
                "INSERT INTO users (id, email, role) VALUES (%s, %s, %s) RETURNING id, ro
                (user_id, email, role)
            )
            new_user = cur.fetchone()
            conn.commit()
            return dict(new_user)
    except Exception as e:
        conn.rollback()
        logger.error(f"Failed to get_or_create_user: {e}")
        raise
    finally:
        conn.close()

def enforce_retention_policy():
    """
    Retention Policy: delete failed builds older than 30 days,
    successful builds older than 90 days.

    Fix #3.5: was mixing manual conn.commit() with the psycopg2 context
    manager, leaving no rollback path on failure. Now uses an explicit
    transaction block with proper commit/rollback.
    """
    logger.info("Running retention policy cleanup...")
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute("""
                DELETE FROM builds
            """)

```

```

        WHERE (status = 'Failed' AND completed_at < NOW() - INTERVAL '30 days')
              OR (status = 'Success' AND completed_at < NOW() - INTERVAL '90 days');
    """
    deleted = cur.rowcount
    conn.commit()
    logger.info(f"Retention policy applied. Deleted {deleted} old builds.")
except Exception as e:
    conn.rollback()
    logger.error(f"Retention policy failed, rolled back: {e}")
    raise
finally:
    conn.close()

def init_db():
    """Run schema.sql to ensure tables exist on startup."""
    try:
        schema_path = os.path.join(os.path.dirname(__file__), "schema.sql")
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        conn = get_connection()
        try:
            with conn.cursor() as cur:
                cur.execute(schema_sql)
            conn.commit()
            logger.info("Database schema initialized successfully.")
        except Exception as e:
            conn.rollback()
            logger.error(f"Failed to execute schema.sql: {e}")
            raise
        finally:
            conn.close()
    except Exception as e:
        logger.error(f"Failed to read/initialize database schema: {e}")

```

File: api\main.py

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\api subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```

"""
ShipZen API Server – Phase 16
FastAPI service that is the sole HTTP entry point for the platform.
All state-changing operations write to PostgreSQL and enqueue to Redis.
The controller and worker drive everything asynchronously from there.
"""

import os
import re
import time
import uuid
import logging
from typing import Optional

import redis as redis_lib
import psycopg2
from psycopg2.extras import DictCursor
from fastapi import FastAPI, HTTPException, Query, Depends, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse, StreamingResponse, Response
import json
from pydantic import BaseModel, field_validator
import boto3
import hmac
import hashlib
import secrets
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from fastapi import Request

```

```

from database import get_connection, init_db
from contextlib import asynccontextmanager
from audit import log_audit_event
from auth import get_current_user, User

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %s',
                    logger = logging.getLogger('api'))

# — Config —————

REDIS_HOST = os.getenv("REDIS_HOST", "redis-master.shipzen-system.svc.cluster.local")
REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))
STREAM_NAME = os.getenv("STREAM_NAME", "deploy_stream")

# ECR repository URL – injected by Terraform at deploy time.
# The API constructs the full image URI as: <ECR_URL>:<deployment_id>
# Users never see or input this value.
ECR_REPOSITORY_URL = os.getenv("ECR_REPOSITORY_URL", "")

# Repo URL allowlist – same pattern used in builder/main.py
_REPO_URL_RE = re.compile(
    r'^(https://[a-zA-Z0-9._:/\:@]+\.\git'
    r'|https://[a-zA-Z0-9._:/\:@]+'
    r'|git@[a-zA-Z0-9._\:-]+:[a-zA-Z0-9._/\-]+\.\git)$'
)

# Kubernetes namespace name rules: lowercase alphanumeric and hyphens, 3-63 chars
_NAMESPACE_RE = re.compile(r'^[a-z0-9][a-z0-9\-\]{1,61}[a-z0-9]$')

# — Redis client —————

def get_redis() -> redis_lib.Redis:
    return redis_lib.Redis(host=REDIS_HOST, port=REDIS_PORT, decode_responses=True)

# — FastAPI app —————

@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield

app = FastAPI(
    title="ShipZen API",
    description="Internal Developer Platform – deploy any repo to Kubernetes",
    version="1.0.0",

```

```

        lifespan=lifespan,
    )

def _user_id_or_ip(request: Request) -> str:
    # Use Authorization header sub if present, fallback to IP
    auth_header = request.headers.get("Authorization")
    if auth_header and auth_header.startswith("Bearer "):
        try:
            token = auth_header.split(" ")[1]
            import jwt
            # For rate limiting, it's cheaper to just decode without full validation
            payload = jwt.decode(token, options={"verify_signature": False})
            if "sub" in payload:
                return payload["sub"]
        except Exception:
            pass
    return get_remote_address(request)

limiter = Limiter(key_func=_user_id_or_ip)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# CORS – allow the Next.js dev server and any deployed UI origin.
# In production, replace "*" with the actual UI domain.
_UI_ORIGINS = list(filter(None, [
    "http://localhost:3000",
    "http://127.0.0.1:3000",
    os.getenv("UI_ORIGIN"),
]))

app.add_middleware(
    CORSMiddleware,
    allow_origins=_UI_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["Authorization", "Content-Type"],
)

# — Request / Response models —————

class CreateProjectRequest(BaseModel):
    name: str
    namespace: str

    @field_validator("namespace")

```

```

    @classmethod
    def validate_namespace(cls, v: str) -> str:
        if not _NAMESPACE_RE.match(v):
            raise ValueError(
                "namespace must be lowercase alphanumeric with hyphens, 3-63 chars, "
                "and cannot start or end with a hyphen"
            )
        return v

class CreateDeploymentRequest(BaseModel):
    repo_url: str
    port: Optional[int] = 8080
    branch: Optional[str] = "main"

    @field_validator("repo_url")
    @classmethod
    def validate_repo_url(cls, v: str) -> str:
        if not _REPO_URL_RE.match(v):
            raise ValueError(
                "repo_url must be an https:// URL or git@host:org/repo.git SSH URL"
            )
        return v

    @field_validator("port")
    @classmethod
    def validate_port(cls, v: int) -> int:
        if v < 1 or v > 65535:
            raise ValueError("port must be between 1 and 65535")
        return v

# — Health —————

@app.get("/healthz", tags=["Health"])
@limiter.limit("100/minute")
def healthz(request: Request):
    """Liveness probe endpoint. Always returns 200."""
    return {"status": "ok"}

# — Projects —————

@app.post("/projects", status_code=201, tags=["Projects"])
@limiter.limit("10/minute")
def create_project(request: Request, body: CreateProjectRequest, current_user: User = Dep
    """

```

```

Create a new project. The controller picks up status=Provisioning
and creates the tenant namespace + RBAC in Kubernetes.
"""

project_id = str(uuid.uuid4())
webhook_secret = secrets.token_hex(32)
try:
    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(
                """
                INSERT INTO projects (id, owner_id, name, namespace, status, webhook_
                VALUES (%s, %s, %s, %s, 'Provisioning', %s)
                RETURNING *;
                """,
                (project_id, current_user.user_id, body.name, body.namespace, webhook_
            )
            project = dict(cur.fetchone())
            conn.commit()
except psycopg2.errors.UniqueViolation:
    raise HTTPException(status_code=409, detail="A project with this ID already exist
except Exception as e:
    logger.error(f"Failed to create project: {e}")
    raise HTTPException(status_code=500, detail="Failed to create project")

log_audit_event(
    project_id=project_id,
    user_id=current_user.user_id,
    action="CREATE",
    resource_type="project",
    resource_id=project_id,
    details={"name": body.name, "namespace": body.namespace},
)
return _serialize(project)

@app.get("/projects", tags=["Projects"])
@limiter.limit("100/minute")
def list_projects(request: Request, current_user: User = Depends(get_current_user)):
    """List all non-deleted (non-Terminating) projects."""
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                if current_user.is_admin:
                    cur.execute(
                        "SELECT * FROM projects WHERE deleted_at IS NULL ORDER BY created

```

```

        )
    else:
        cur.execute(
            "SELECT * FROM projects WHERE deleted_at IS NULL AND owner_id = %
            (current_user.user_id,)
        )
        return [_serialize(dict(r)) for r in cur.fetchall()]
except Exception as e:
    logger.error(f"Failed to list projects: {e}")
    raise HTTPException(status_code=500, detail="Failed to list projects")

@app.get("/projects/{project_id}", tags=["Projects"])
@limiter.limit("100/minute")
def get_project(request: Request, project_id: str, current_user: User = Depends(get_curre
    """Get a single project by ID."""
    project = _get_project_or_404(project_id, current_user)
    return _serialize(project)

@app.delete("/projects/{project_id}", status_code=202, tags=["Projects"])
@limiter.limit("10/minute")
def delete_project(request: Request, project_id: str, current_user: User = Depends(get_cu
    """
    Soft-delete a project – sets status to Terminating.
    The controller will delete the Kubernetes namespace and then
    hard-delete the row once the namespace is gone.
    """
    _get_project_or_404(project_id, current_user)
    try:
        with get_connection() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    "UPDATE projects SET status = 'Terminating', deleted_at = NOW() WHERE
                    (project_id,),
                )
                conn.commit()
    except Exception as e:
        logger.error(f"Failed to delete project {project_id}: {e}")
        raise HTTPException(status_code=500, detail="Failed to delete project")

    log_audit_event(
        project_id=project_id,
        user_id=current_user.user_id,
        action="DELETE",

```



```

        resource_type="project",
        resource_id=project_id,
        details={},
    )
    return {"message": f"Project {project_id} marked for termination"}

class AnalyzeRequest(BaseModel):
    repo_url: str
    branch: str = "main"

@app.post("/projects/analyze", tags=["Projects"])
@limiter.limit("5/minute")
def analyze_repo(request: Request, body: AnalyzeRequest, current_user: User = Depends(get_current_user)):
    """Analyze a Git repository and detect deployable services."""
    import tempfile
    import subprocess
    from analyzer import RepoAnalyzer

    with tempfile.TemporaryDirectory() as tmpdir:
        try:
            # Shallow clone
            subprocess.run(
                ["git", "clone", "--depth", "1", "-b", body.branch, body.repo_url, tmpdir],
                check=True, capture_output=True, timeout=30
            )
        except Exception as e:
            logger.error(f"Failed to clone repo for analysis: {e}")
            raise HTTPException(status_code=400, detail="Failed to clone repository")

        analyzer = RepoAnalyzer(repo_path=tmpdir, repo_name=body.repo_url.split('/')[-1])
        services = analyzer.analyze()

    return {"services": [s.__dict__ for s in services]}

# — Deployments —————

@app.post("/projects/{project_id}/deployments", status_code=202, tags=["Deployments"])
@limiter.limit("5/minute")
def create_deployment(request: Request, project_id: str, body: CreateDeploymentRequest, current_user: User = Depends(get_current_user)):
    """
    Submit a deployment request. Only a repo URL is required.
    - The platform generates the image URI automatically from ECR_REPOSITORY_URL.
    - Scaling is handled by Karpenter/KEDA – the user does not set replicas.
    - Port defaults to 8080; override only if your app listens elsewhere.
    """

```

```

project = _get_project_or_404(project_id, current_user)
if project["status"] not in ("Ready", "Provisioning"):
    raise HTTPException(
        status_code=409,
        detail=f"Project is in status '{project['status']}' and cannot accept deployment"
    )

deployment_id = str(uuid.uuid4())
queued_at = str(time.time())

# Build the image URI – users never input this.
# Format: <ecr_repo_url>:<deployment_id>
# deployment_id as tag gives a unique, traceable, immutable image per deploy.
if ECR_REPOSITORY_URL:
    # Base registry e.g. 123456789012.dkr.ecr.region.amazonaws.com
    base_registry = ECR_REPOSITORY_URL.split("/")[0]
    image_uri = f"{base_registry}/shipzen-builds/{project_id}:{deployment_id}"
else:
    # Local dev / testing fallback – no ECR configured
    image_uri = f"local/shipzen-builds/{project_id}:{deployment_id}"

try:
    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(
                """
                INSERT INTO deployments
                (deployment_id, project_id, repo_url, image_uri, replicas, port,
                VALUES (%s, %s, %s, %s, %s, %s, 'Queued')
                RETURNING *;
                """,
                (
                    deployment_id, project_id,
                    body.repo_url, image_uri,
                    1,          # Platform controls scaling – initial replica count
                    body.port,
                ),
            )
            deployment = dict(cur.fetchone())
            conn.commit()
except Exception as e:
    logger.error(f"Failed to create deployment: {e}")
    raise HTTPException(status_code=500, detail="Failed to create deployment")

# Enqueue to Redis stream – worker picks this up and hands off to builder

```

```

try:
    r = get_redis()
    r.xadd(STREAM_NAME, {
        "deployment_id": deployment_id,
        "project_id":    project_id,
        "repo_url":      body.repo_url,
        "branch":        body.branch,
        "image_name":    image_uri,
        "queued_at":     queued_at,
        "retries":       "0",
    })
except Exception as e:
    logger.error(f"Failed to enqueue deployment {deployment_id}: {e}")
    try:
        with get_connection() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    "UPDATE deployments SET state = 'Failed', last_error = %s WHERE d"
                    ("Failed to enqueue to Redis", deployment_id),
                )
            conn.commit()
    except Exception:
        pass
    raise HTTPException(status_code=500, detail="Failed to enqueue deployment")

log_audit_event(
    project_id=project_id,
    user_id=current_user.user_id,
    action="DEPLOY",
    resource_type="deployment",
    resource_id=deployment_id,
    details={"repo_url": body.repo_url, "branch": body.branch},
)
return _serialize(deployment)

@app.post("/projects/{project_id}/rollback", status_code=202, tags=["Deployments"])
@limiter.limit("5/minute")
def rollback_deployment(request: Request, project_id: str, current_user: User = Depends(g
    """Re-deploy the last known-good image without rebuilding."""
    project = _get_project_or_404(project_id, current_user)

    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            # Find the last successful deployment

```

```

        cur.execute("""
            SELECT * FROM deployments
            WHERE project_id = %s AND state = 'Running'
            ORDER BY updated_at DESC LIMIT 1;
        """, (project_id,))
        last_good = cur.fetchone()

        if not last_good:
            raise HTTPException(status_code=409, detail="No previous successful deplo

        deployment_id = str(uuid.uuid4())
        cur.execute("""
            INSERT INTO deployments
                (deployment_id, project_id, repo_url, image_uri, replicas, port, stat
            VALUES (%s, %s, %s, %s, %s, %s, 'Deploying')
            RETURNING *;
        """, (
            deployment_id, project_id, last_good['repo_url'],
            last_good['image_uri'], last_good['replicas'], last_good['port']
        ))
        new_dep = dict(cur.fetchone())
        conn.commit()

# Publish state update to Redis
try:
    r = get_redis()
    r.publish(f"shipzen:status:{deployment_id}", json.dumps({"state": "Deploying", "l
except Exception as e:
    logger.warning(f"Failed to publish status to Redis: {e}")

log_audit_event(
    project_id=project_id,
    user_id=current_user.user_id,
    action="ROLLBACK",
    resource_type="deployment",
    resource_id=deployment_id,
    details={"from_image": last_good['image_uri']},
)
return {"message": "Rollback queued", "deployment_id": deployment_id, "status": "Depl

@app.get("/projects/{project_id}/deployments", tags=["Deployments"])
@limiter.limit("100/minute")
def list_deployments(
    request: Request,

```

```

project_id: str,
limit: int = Query(default=20, ge=1, le=100),
cursor: Optional[str] = Query(default=None, description="cursor format: <updated_at>|
current_user: User = Depends(get_current_user),
):
    """
    List deployments for a project with keyset pagination.
    Pass the `<updated_at>|<deployment_id>` value of the last item as `cursor` to get the
    """
    _get_project_or_404(project_id, current_user)

    query = """
        SELECT deployment_id, project_id, repo_url, image_uri, replicas, port, state, upd
        FROM deployments
        WHERE project_id = %s
    """
    params = [project_id]

    if cursor:
        try:
            cursor_updated_at, cursor_deployment_id = cursor.split("|", 1)
            query += " AND (updated_at, deployment_id) < (%s, %s)"
            params.extend([cursor_updated_at, cursor_deployment_id])
        except ValueError:
            raise HTTPException(status_code=400, detail="Invalid cursor format")

    query += " ORDER BY updated_at DESC, deployment_id DESC LIMIT %s;"
    params.append(limit)

    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute(query, tuple(params))
                return [_serialize(dict(r)) for r in cur.fetchall()]
    except Exception as e:
        logger.error(f"Failed to list deployments: {e}")
        raise HTTPException(status_code=500, detail="Failed to list deployments")

@app.get("/projects/{project_id}/deployments/{deployment_id}", tags=["Deployments"])
@limiter.limit("100/minute")
def get_deployment(request: Request, project_id: str, deployment_id: str, current_user: U
    """Get a single deployment by ID."""
    _get_project_or_404(project_id, current_user)
    deployment = _get_deployment_or_404(project_id, deployment_id)

```

```

    return _serialize(deployment)

@app.websocket("/ws/projects/{project_id}/deployments/{deployment_id}/status")
async def websocket_deployment_status(websocket: WebSocket, project_id: str, deployment_id: str):
    if not token:
        await websocket.close(code=1008)
        return
    from auth import get_current_user_from_token
    try:
        user = get_current_user_from_token(token)
    except Exception:
        await websocket.close(code=1008)
        return

    await websocket.accept()

    import redis.asyncio as aioredis
    import asyncio
    r = aioredis.Redis(host=REDIS_HOST, port=REDIS_PORT, decode_responses=True)
    pubsub = r.pubsub()
    await pubsub.subscribe(f"shipzen:status:{deployment_id}")

    def fetch_initial():
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute(
                    "SELECT state, last_error FROM deployments WHERE deployment_id = %s AND project_id = %s"
                    (deployment_id, project_id),
                )
                return cur.fetchone()

    try:
        row = await asyncio.to_thread(fetch_initial)
        if row:
            await websocket.send_json({"state": row['state'], "last_error": row['last_error']})

        async for message in pubsub.listen():
            if message["type"] == "message":
                data = json.loads(message["data"])
                await websocket.send_json(data)
                if data.get("state") in ("Running", "Failed", "DLQ"):
                    break
    except WebSocketDisconnect:
        logger.info(f"WebSocket disconnected for {deployment_id}")
    except Exception as e:

```

```

        logger.error(f"WS error: {e}")
    finally:
        await pubsub.unsubscribe()
        await r.aclose()

@app.get("/projects/{project_id}/deployments/{deployment_id}/logs/stream", tags=["Deploym
async def stream_logs(project_id: str, deployment_id: str, current_user: User = Depends(g
    _get_project_or_404(project_id, current_user)

import redis.asyncio as aioredis
r = aioredis.Redis(host=REDIS_HOST, port=REDIS_PORT, decode_responses=True)

async def event_stream():
    pubsub = r.pubsub()
    await pubsub.subscribe(f"shipzen:logs:{deployment_id}")
    try:
        async for message in pubsub.listen():
            if message["type"] == "message":
                yield f"data: {message['data']}\n\n"
    finally:
        await pubsub.unsubscribe()
        await r.aclose()

return StreamingResponse(
    event_stream(),
    media_type="text/event-stream",
    headers={"Cache-Control": "no-cache", "Connection": "keep-alive"}
)

@app.websocket("/ws/projects/{project_id}/deployments/{deployment_id}/logs")
async def websocket_deployment_logs(
    websocket: WebSocket,
    project_id: str,
    deployment_id: str,
    token: str = Query(None),
):
    """
    WebSocket endpoint for live build log streaming.
    Subscribes to the Redis Pub/Sub channel `shipzen:logs:{deployment_id}`
    and forwards each line to the connected client as plain text.
    Auth is passed via ?token= query param (same pattern as the status WS).
    """
    if not token:

```

```

        await websocket.close(code=1008)
    return

from auth import get_current_user_from_token
try:
    user = get_current_user_from_token(token)
except Exception:
    await websocket.close(code=1008)
    return

# Verify the deployment belongs to this project
try:
    import asyncio
    def verify():
        _get_project_or_404(project_id, user)
        _get_deployment_or_404(project_id, deployment_id)
    await asyncio.to_thread(verify)
except HTTPException:
    await websocket.close(code=1008)
    return

await websocket.accept()

import redis.asyncio as aioredis
r = aioredis.Redis(host=REDIS_HOST, port=REDIS_PORT, decode_responses=True)
pubsub = r.pubsub()
await pubsub.subscribe(f"shipzen:logs:{deployment_id}")

try:
    async for message in pubsub.listen():
        if message["type"] == "message":
            await websocket.send_text(message["data"])
except WebSocketDisconnect:
    logger.info(f"Log WS disconnected for {deployment_id}")
except Exception as e:
    logger.error(f"Log WS error for {deployment_id}: {e}")
finally:
    await pubsub.unsubscribe()
    await r.aclose()

# — Builds —————

@app.get("/projects/{project_id}/deployments/{deployment_id}/builds", tags=["Builds"])
@limiter.limit("100/minute")
def list_builds(request: Request, project_id: str, deployment_id: str, current_user: User)
    """List all builds for a deployment, most recent first."""

```



```

_get_project_or_404(project_id, current_user)
_get_deployment_or_404(project_id, deployment_id)

try:
    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(
                """
                SELECT build_id, deployment_id, s3_log_uri, status, started_at, compl
                FROM builds
                WHERE deployment_id = %s
                ORDER BY started_at DESC;
                """,
                (deployment_id,),
            )
            return [_serialize(dict(r)) for r in cur.fetchall()]
except Exception as e:
    logger.error(f"Failed to list builds: {e}")
    raise HTTPException(status_code=500, detail="Failed to list builds")

@app.get("/projects/{project_id}/deployments/{deployment_id}/builds/{build_id}/logs", tag
@limiter.limit("100/minute")
def get_build_logs(request: Request, project_id: str, deployment_id: str, build_id: str,
    """Stream build log content directly, proxied through the API to avoid S3 CORS issues
    _get_project_or_404(project_id, current_user)
    _get_deployment_or_404(project_id, deployment_id)

try:
    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(
                "SELECT s3_log_uri FROM builds WHERE build_id = %s AND deployment_id
                (build_id, deployment_id)
            )
            row = cur.fetchone()
            if not row or not row["s3_log_uri"]:
                raise HTTPException(status_code=404, detail="Log not found")

            s3_uri = row["s3_log_uri"]
            if not s3_uri.startswith("s3://"):
                raise HTTPException(status_code=400, detail="Invalid log URI")

            bucket = s3_uri.split("/")[2]
            key = "/" + s3_uri.split("/")[3:]

```

```

        if not bucket:
            raise HTTPException(status_code=404, detail="Log storage not configur

s3 = boto3.client('s3')
try:
    obj = s3.get_object(Bucket=bucket, Key=key)
except s3.exceptions.NoSuchKey:
    raise HTTPException(status_code=404, detail="Log file not found in S3

content = obj['Body'].read()
return Response(
    content=content,
    media_type="text/plain",
    headers={"Content-Disposition": f"inline; filename=build-{build_id[:8]
    )
except HTTPException:
    raise
except Exception as e:
    logger.error(f"Failed to fetch build log: {e}")
    raise HTTPException(status_code=500, detail="Failed to get logs")

# — Audit —————

@app.get("/projects/{project_id}/audit", tags=["Audit"])
@limiter.limit("100/minute")
def get_audit_logs(
    request: Request,
    project_id: str,
    limit: int = Query(default=50, ge=1, le=500),
    current_user: User = Depends(get_current_user),
):
    """List audit log entries for a project."""
    _get_project_or_404(project_id, current_user)

    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute(
                    """
                    SELECT * FROM audit_logs
                    WHERE project_id = %s
                    ORDER BY timestamp DESC
                    LIMIT %s;
                    """,
                    (project_id, limit),

```

```

        )
        return [_serialize(dict(r)) for r in cur.fetchall()]
except Exception as e:
    logger.error(f"Failed to fetch audit logs: {e}")
    raise HTTPException(status_code=500, detail="Failed to fetch audit logs")

@app.get("/audit", tags=["Audit"])
@limiter.limit("100/minute")
def get_global_audit_logs(
    request: Request,
    limit: int = Query(default=20, ge=1, le=100),
    current_user: User = Depends(get_current_user),
):
    """List recent audit log entries across all projects owned by the user."""
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                if current_user.is_admin:
                    cur.execute(
                        """
                        SELECT a.*, p.name as project_name
                        FROM audit_logs a
                        JOIN projects p ON a.project_id = p.id
                        ORDER BY a.timestamp DESC LIMIT %s;
                        """,
                        (limit,),
                    )
                else:
                    cur.execute(
                        """
                        SELECT a.*, p.name as project_name
                        FROM audit_logs a
                        JOIN projects p ON a.project_id = p.id
                        WHERE a.user_id = %s OR p.owner_id = %s
                        ORDER BY a.timestamp DESC LIMIT %s;
                        """,
                        (current_user.user_id, current_user.user_id, limit),
                    )
                return [_serialize(dict(r)) for r in cur.fetchall()]
    except Exception as e:
        logger.error(f"Failed to fetch global audit logs: {e}")
        raise HTTPException(status_code=500, detail="Failed to fetch audit logs")

```

— Env Vars

```

@app.get("/projects/{project_id}/env", tags=["Environment"])
@limiter.limit("100/minute")
def get_env_vars(request: Request, project_id: str, current_user: User = Depends(get_curr
    project = _get_project_or_404(project_id, current_user)
    secret_id = f"shipzen/{project['name']}/"
    sm = boto3.client('secretsmanager')
    try:
        # We only return the keys, not the values for security
        res = sm.get_secret_value(SecretId=secret_id)
        import json
        secret_dict = json.loads(res.get('SecretString', '{}'))
        return {"keys": list(secret_dict.keys())}
    except sm.exceptions.ResourceNotFoundException:
        return {"keys": []}
    except Exception as e:
        logger.error(f"Failed to fetch env vars for {project_id}: {e}")
        raise HTTPException(status_code=500, detail="Failed to fetch env vars")

@app.put("/projects/{project_id}/env", tags=["Environment"])
@limiter.limit("20/minute")
def put_env_var(request: Request, project_id: str, body: dict, current_user: User = Depen
    """Expected body: {"key": "API_KEY", "value": "secret123"}"""
    project = _get_project_or_404(project_id, current_user)
    key = body.get("key")
    value = body.get("value")
    if not key or not value:
        raise HTTPException(status_code=400, detail="Missing key or value")

    secret_id = f"shipzen/{project['name']}/"
    sm = boto3.client('secretsmanager')
    import json

    try:
        try:
            res = sm.get_secret_value(SecretId=secret_id)
            secret_dict = json.loads(res.get('SecretString', '{}'))
        except sm.exceptions.ResourceNotFoundException:
            secret_dict = {}

        secret_dict[key] = value

    try:
        sm.update_secret(SecretId=secret_id, SecretString=json.dumps(secret_dict))
    except sm.exceptions.ResourceNotFoundException:
        sm.create_secret(Name=secret_id, SecretString=json.dumps(secret_dict))

```

```

    log_audit_event(
        project_id=project_id,
        user_id=current_user.user_id,
        action="UPDATE_ENV",
        resource_type="project",
        resource_id=project_id,
        details={"key": key},
    )
    return {"message": "Updated successfully"}
except Exception as e:
    logger.error(f"Failed to update env var for {project_id}: {e}")
    raise HTTPException(status_code=500, detail="Failed to update env var")

@app.delete("/projects/{project_id}/env/{key}", tags=["Environment"])
@limiter.limit("20/minute")
def delete_env_var(request: Request, project_id: str, key: str, current_user: User = Depends(get_current_user)):
    project = _get_project_or_404(project_id, current_user)
    secret_id = f"shipzen/{project['name']}/"
    sm = boto3.client('secretsmanager')
    import json

    try:
        res = sm.get_secret_value(SecretId=secret_id)
        secret_dict = json.loads(res.get('SecretString', '{}'))
        if key in secret_dict:
            del secret_dict[key]
            sm.update_secret(SecretId=secret_id, SecretString=json.dumps(secret_dict))

            log_audit_event(
                project_id=project_id,
                user_id=current_user.user_id,
                action="DELETE_ENV",
                resource_type="project",
                resource_id=project_id,
                details={"key": key},
            )
            return {"message": "Deleted successfully"}
    except sm.exceptions.ResourceNotFoundException:
        return {"message": "Deleted successfully"}
    except Exception as e:
        logger.error(f"Failed to delete env var for {project_id}: {e}")
        raise HTTPException(status_code=500, detail="Failed to delete env var")

```

```

@app.post("/webhooks/github/{project_id}", tags=["Webhooks"])
@limiter.limit("60/minute")
async def github_webhook(request: Request, project_id: str):
    # Verify signature
    signature_header = request.headers.get("x-hub-signature-256")
    if not signature_header:
        raise HTTPException(status_code=400, detail="Missing signature")

    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute("SELECT webhook_secret FROM projects WHERE id = %s;", (project_id,))
                row = cur.fetchone()
                if not row or not row["webhook_secret"]:
                    raise HTTPException(status_code=404, detail="Webhook secret not found")
                webhook_secret = row["webhook_secret"]
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Failed to get webhook secret: {e}")
        raise HTTPException(status_code=500, detail="Database error")

    # Validation logic requires raw body
    body_bytes = await request.body()
    expected_mac = hmac.new(webhook_secret.encode(), body_bytes, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(f"sha256={expected_mac}", signature_header):
        raise HTTPException(status_code=401, detail="Invalid signature")

    import json
    try:
        payload = json.loads(body_bytes)
    except json.JSONDecodeError:
        raise HTTPException(status_code=400, detail="Invalid JSON body")

    # Trigger deploy
    # Determine branch
    branch = "main"
    if "ref" in payload:
        branch = payload["ref"].split("/")[-1]

    # We need repo URL
    repo_url = payload.get("repository", {}).get("clone_url")
    if not repo_url:
        raise HTTPException(status_code=400, detail="Missing repository clone_url")

```

```

deployment_id = str(uuid.uuid4())
queued_at = str(time.time())
if ECR_REPOSITORY_URL:
    base_registry = ECR_REPOSITORY_URL.split("/")[0]
    image_uri = f"{base_registry}/shipzen-builds/{project_id}:{deployment_id}"
else:
    image_uri = f"local/shipzen-builds/{project_id}:{deployment_id}"

try:
    with get_connection() as conn:
        with conn.cursor(cursor_factory=DictCursor) as cur:
            cur.execute(
                "SELECT repo_url, port FROM deployments WHERE project_id = %s ORDER BY"
                "(project_id,)"
            )
            last_deploy = cur.fetchone()

            if not last_deploy:
                raise HTTPException(status_code=400, detail="Project has no existing")

            if last_deploy["repo_url"] != repo_url:
                raise HTTPException(status_code=403, detail="Webhook repository does")

            port = last_deploy["port"]

            cur.execute(
                """
                INSERT INTO deployments (deployment_id, project_id, repo_url, image_u
                VALUES (%s, %s, %s, %s, %s, %s, 'Queued')
                """,
                (deployment_id, project_id, repo_url, image_uri, 1, port)
            )
            conn.commit()

    r = get_redis()
    r.xadd(STREAM_NAME, {
        "deployment_id": deployment_id,
        "project_id": project_id,
        "repo_url": repo_url,
        "branch": branch,
        "image_name": image_uri,
        "queued_at": queued_at,
        "retries": "0",
    })

```

```

except Exception as e:
    logger.error(f"Failed to process webhook for {project_id}: {e}")
    raise HTTPException(status_code=500, detail="Failed to process webhook")

log_audit_event(
    project_id=project_id,
    user_id="webhook",
    action="WEBHOOK_DEPLOY",
    resource_type="deployment",
    resource_id=deployment_id,
    details={"repo_url": repo_url, "branch": branch},
)

return {"message": "Deployment triggered", "deployment_id": deployment_id}

# — Users & Admin —————

@app.get("/users/me", tags=["Users"])
@limiter.limit("100/minute")
def get_me(request: Request, current_user: User = Depends(get_current_user)):
    """Get the currently logged-in user's profile and role."""
    return {"user_id": current_user.user_id, "is_admin": current_user.is_admin}

class UpdateRoleRequest(BaseModel):
    role: str

@app.get("/admin/users", tags=["Admin"])
@limiter.limit("100/minute")
def list_users(request: Request, current_user: User = Depends(get_current_user)):
    if not current_user.is_admin:
        raise HTTPException(status_code=403, detail="Forbidden")
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute("SELECT id, email, role, created_at FROM users ORDER BY created_at")
                return [_serialize(dict(r)) for r in cur.fetchall()]
    except Exception as e:
        logger.error(f"Failed to list users: {e}")
        raise HTTPException(status_code=500, detail="Database error")

@app.put("/admin/users/{user_id}/role", tags=["Admin"])
@limiter.limit("50/minute")
def update_user_role(request: Request, user_id: str, body: UpdateRoleRequest, current_user: User = Depends(get_current_user)):
    """Update the role of a user. Only admin can update roles.
    """
    if not current_user.is_admin:
        raise HTTPException(status_code=403, detail="Forbidden")
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute("SELECT id, email, role, created_at FROM users WHERE id = %s", (user_id,))
                user = cur.fetchone()
                if user is None:
                    raise HTTPException(status_code=404, detail="User not found")
                cur.execute("UPDATE users SET role = %s WHERE id = %s", (body.role, user_id))
                conn.commit()
    except Exception as e:
        logger.error(f"Failed to update user role: {e}")
        raise HTTPException(status_code=500, detail="Database error")

```



```

if not current_user.is_admin:
    raise HTTPException(status_code=403, detail="Forbidden")
if body.role not in ["admin", "user"]:
    raise HTTPException(status_code=400, detail="Invalid role. Must be 'admin' or 'us

try:
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("UPDATE users SET role = %s WHERE id = %s RETURNING id;", (bo
            if not cur.fetchone():
                raise HTTPException(status_code=404, detail="User not found")
            conn.commit()
except HTTPException:
    raise
except Exception as e:
    logger.error(f"Failed to update user role: {e}")
    raise HTTPException(status_code=500, detail="Database error")

log_audit_event(
    project_id=None,
    user_id=current_user.user_id,
    action="UPDATE_ROLE",
    resource_type="user",
    resource_id=user_id,
    details={"new_role": body.role},
)
return {"message": f"User {user_id} role updated to {body.role}"}

@app.get("/admin/audit-logs", tags=["Admin"])
@limiter.limit("100/minute")
def list_global_audit_logs(request: Request, limit: int = Query(50, le=200), current_user
    if not current_user.is_admin:
        raise HTTPException(status_code=403, detail="Forbidden")
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute(
                    "SELECT * FROM audit_logs ORDER BY timestamp DESC LIMIT %s;",
                    (limit,)
                )
                return [_serialize(dict(r)) for r in cur.fetchall()]
    except Exception as e:
        logger.error(f"Failed to fetch global audit logs: {e}")
        raise HTTPException(status_code=500, detail="Database error")

```

— Helpers

```

def _get_project_or_404(project_id: str, current_user: User) -> dict:
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute("SELECT * FROM projects WHERE id = %s;", (project_id,))
                row = cur.fetchone()
    except Exception as e:
        logger.error(f"DB error fetching project {project_id}: {e}")
        raise HTTPException(status_code=500, detail="Database error")

    if not row:
        raise HTTPException(status_code=404, detail=f"Project '{project_id}' not found")

    if not current_user.is_admin and row["owner_id"] != current_user.user_id:
        raise HTTPException(status_code=403, detail="Forbidden")

    return dict(row)

def _get_deployment_or_404(project_id: str, deployment_id: str) -> dict:
    try:
        with get_connection() as conn:
            with conn.cursor(cursor_factory=DictCursor) as cur:
                cur.execute(
                    "SELECT * FROM deployments WHERE deployment_id = %s AND project_id = %s",
                    (deployment_id, project_id),
                )
                row = cur.fetchone()
    except Exception as e:
        logger.error(f"DB error fetching deployment {deployment_id}: {e}")
        raise HTTPException(status_code=500, detail="Database error")

    if not row:
        raise HTTPException(status_code=404, detail=f"Deployment '{deployment_id}' not found")

    return dict(row)

def _serialize(obj: dict) -> dict:
    """Convert non-JSON-serializable types (datetime) to strings."""
    return {
        k: v.isoformat() if hasattr(v, "isoformat") else v
        for k, v in obj.items()
    }

```

}

6. Infrastructure Manifests Deep Dive

File: `infra\kustomization.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\infra` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

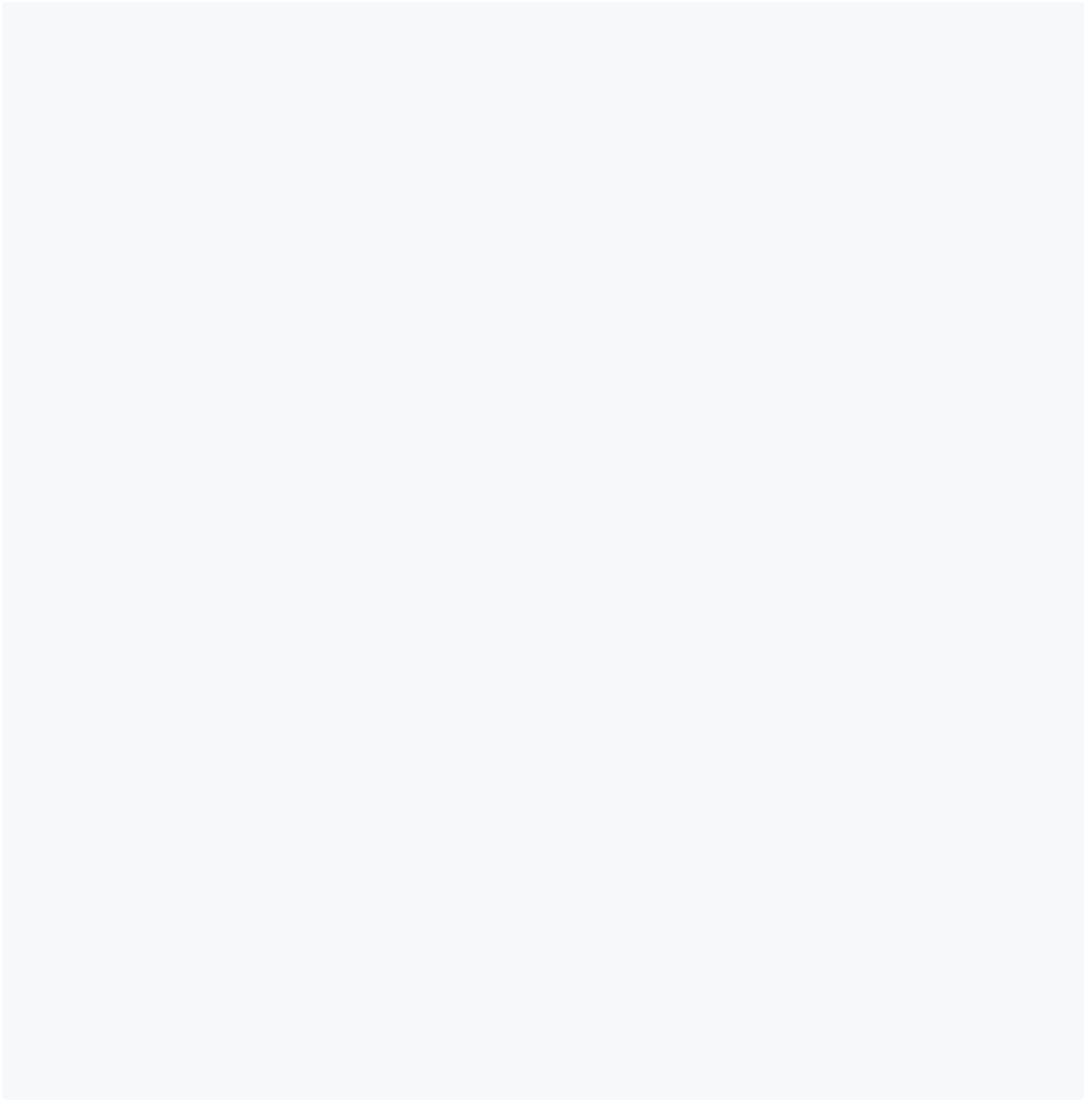
Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
# Top-level kustomization for ArgoCD.
# ArgoCD syncs path: infra which resolves to this file.
# Every sub-directory that has a kustomization.yaml is listed here.
resources:
  - system/
  - controller/
  - worker/
  - api/
  - scale/
  - ui/
images:
  - name: shipzen-api
    newName: ghcr.io/jeneeldumasia/shipzen-api
    newTag: sha-9bcf230
```

```
- name: shipzen-controller  
  newName: ghcr.io/jeneeldumasia/shipzen-controller  
  newTag: sha-9bcf230
```

- name: shipzen-worker
newName: ghcr.io/jeneeldumasia/shipzen-worker
newTag: sha-9bcf230

- name: shipzen-builder
newName: ghcr.io/jeneeldumasia/shipzen-builder
newTag: sha-88e93b4
- name: shipzen-ui
newName: ghcr.io/jeneeldumasia/shipzen-ui
newTag: sha-9bcf230



File: infra\api\deployment.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shipzen-api
  namespace: shipzen-system
  labels:
    app: shipzen-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: shipzen-api
  template:
    metadata:
      labels:
        app: shipzen-api
    spec:
      serviceAccountName: shipzen-api-sa
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: api
          image: shipzen-api:latest
          imagePullPolicy: Always
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
```



```

ports:
  - name: http
    containerPort: 8000
    protocol: TCP
env:
  - name: AWS_ROLE_ARN
    value: "arn:aws:iam::952994886652:role/ShipZenBuilderRole"
  - name: AWS_WEB_IDENTITY_TOKEN_FILE
    value: "/var/run/secrets/eks.amazonaws.com/serviceaccount/token"
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: shipzen-db-credentials
        key: url
  - name: REDIS_HOST
    value: "redis-master.shipzen-system.svc.cluster.local"
  - name: STREAM_NAME
    value: "deploy_stream"
  - name: UI_ORIGIN
    value: "https://shipzen.jeneeldumasia.codes"
  - name: ECR_REPOSITORY_URL
    valueFrom:
      secretKeyRef:
        name: shipzen-ecr-config
        key: repository_url
  - name: GITHUB_ENABLED
    valueFrom:
      secretKeyRef:
        name: shipzen-github
        key: enabled
        optional: true
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
livenessProbe:
  httpGet:
    path: /healthz
    port: 8000
  initialDelaySeconds: 10
  periodSeconds: 15
readinessProbe:

```

```
    httpGet:
      path: /healthz
      port: 8000
      initialDelaySeconds: 5
      periodSeconds: 10
    volumeMounts:
      - mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount
        name: aws-iam-token
        readOnly: true
  volumes:
    - name: aws-iam-token
      projected:
        defaultMode: 420
        sources:
          - serviceAccountToken:
              audience: sts.amazonaws.com
              expirationSeconds: 86400
              path: token
```

File: infra\api\httproute.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shipzen-api
  namespace: shipzen-system
spec:
  hostnames:
  - "shipzen.jeneeldumasia.codes"
  parentRefs:
  - name: shipzen-gateway
    namespace: shipzen-system
    sectionName: https
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api/v1
    filters:
    - type: URLRewrite
      urlRewrite:
        path:
          type: ReplacePrefixMatch
          replacePrefixMatch: /
  backendRefs:
  - name: shipzen-api
    port: 80
```

File: infra\api\kustomization.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml
  - serviceaccount.yaml
  - httproute.yaml
  - networkpolicy.yaml
```

File: infra\api\networkpolicy.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-server-egress
  namespace: shipzen-system
spec:
  podSelector:
    matchLabels:
      app: shipzen-api
  policyTypes:
    - Egress
  egress:
    # Allow DNS
    - ports:
        - port: 53
          protocol: UDP
        - port: 53
          protocol: TCP
    # Allow PostgreSQL
    - ports:
        - port: 5432
          protocol: TCP
      to:
        - podSelector:
            matchLabels:
              app.kubernetes.io/name: postgresql
    # Allow Redis
    - ports:
        - port: 6379
          protocol: TCP
      to:
```

```
- podSelector:
  matchLabels:
    app.kubernetes.io/name: redis
# Allow HTTPS to external APIs (AWS Secrets Manager, ECR, Auth0)
- ports:
  - port: 443
    protocol: TCP
to:
  - ipBlock:
    cidr: 0.0.0.0/0
    except:
      - 10.0.0.0/8
      - 172.16.0.0/12
      - 192.168.0.0/16
```

File: infra\api\service.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: Service
metadata:
  name: shipzen-api
  namespace: shipzen-system
  labels:
    app: shipzen-api
spec:
  type: ClusterIP
  selector:
    app: shipzen-api
  ports:
    - name: http
      port: 80
      targetPort: 8000
      protocol: TCP
```

File: `infra\api\serviceaccount.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\infra` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: shipzen-api-sa
  namespace: shipzen-system
  annotations:
    eks.amazonaws.com/role-arn: "arn:aws:iam::952994886652:role/ShipZenBuilderRole"
automountServiceAccountToken: true
```


File: infra\controller\clusterrole.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
# The controller needs cluster-wide permissions to create/delete namespaces
# and manage resources in tenant namespaces it doesn't own at creation time.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: shipzen-controller
rules:
  - apiGroups: [""]
    resources: ["namespaces"]
    verbs: ["get", "list", "watch", "create", "delete"]
  - apiGroups: [""]
    resources: ["serviceaccounts", "resourcequotas", "limitranges", "services", "configmaps"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["networking.k8s.io"]
    resources: ["networkpolicies"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["gateway.networking.k8s.io"]
    resources: ["httproutes"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["rbac.authorization.k8s.io"]
    resources: ["roles", "rolebindings"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["external-secrets.io"]
    resources: ["externalsecrets"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["policy"]
    resources: ["poddisruptionbudgets"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: shipzen-controller
subjects:
  - kind: ServiceAccount
    name: shipzen-controller-sa
    namespace: shipzen-system
roleRef:
  kind: ClusterRole
  name: shipzen-controller
  apiGroup: rbac.authorization.k8s.io
```

File: infra\controller\deployment.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shipzen-controller
  namespace: shipzen-system
  labels:
    app: shipzen-controller
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shipzen-controller
  template:
    metadata:
      labels:
        app: shipzen-controller
    spec:
      serviceAccountName: shipzen-controller-sa
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: controller
          image: shipzen-controller:latest
          imagePullPolicy: Always
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
```

```
ports:
  - name: metrics
    containerPort: 9090
    protocol: TCP
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: shipzen-db-credentials
        key: url
  - name: RECONCILIATION_INTERVAL
    value: "60"
  - name: ECR_REGISTRY
    valueFrom:
      secretKeyRef:
        name: shipzen-ecr-config
        key: registry_hostname
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
livenessProbe:
  httpGet:
    path: /metrics
    port: 9090
  initialDelaySeconds: 15
  periodSeconds: 20
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /metrics
    port: 9090
  initialDelaySeconds: 10
  periodSeconds: 10
```

File: infra\controller\kustomization.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - serviceaccount.yaml
  - clusterrole.yaml
  - deployment.yaml
```

File: infra\controller\serviceaccount.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: shipzen-controller-sa
  namespace: shipzen-system
automountServiceAccountToken: true # Controller needs K8s API access to reconcile resource
```



File: infra\scale\hpa.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shipzen-controller-hpa
  namespace: shipzen-system
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shipzen-controller
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 80
```

File: infra\scale\karpenter.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
# Upgraded from v1beta1 to v1 (stable) – required for Karpenter >= 1.0
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
  name: shipzen-default
spec:
  # v1 requires amiSelectorTerms instead of amiFamily
  amiSelectorTerms:
    - alias: al2023@latest
  role: "ShipZenKarpenterNodeRole"
  subnetSelectorTerms:
    - tags:
        karpenter.sh/discovery: "ShipZen"
  securityGroupSelectorTerms:
    - tags:
        karpenter.sh/discovery: "ShipZen"
---
# Builder NodePool – dedicated to build jobs, tainted to prevent other workloads
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: builder-pool
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64"]
        - key: karpenter.sh/capacity-type
          operator: In
```



```

        values: ["spot", "on-demand"]
      - key: karpenter.k8s.aws/instance-category
        operator: In
        values: ["c", "m", "r", "t"]
    nodeClassRef:
      group: karpenter.k8s.aws
      kind: EC2NodeClass
      name: shipzen-default
    taints:
      - key: shipzen.jeneeldumasia.codes/dedicated
        value: builder
        effect: NoSchedule
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 1m
  limits:
    cpu: "4"
    memory: "8Gi"
---
# Tenant NodePool – stable on-demand nodes for user application workloads
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: tenant-pool
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64"]
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["on-demand"]
        - key: karpenter.k8s.aws/instance-category
          operator: In
          values: ["m", "c", "t"]
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: shipzen-default
      taints:
        - key: shipzen.jeneeldumasia.codes/dedicated
          value: tenant
          effect: NoSchedule

```

```
disruption:
  consolidationPolicy: WhenEmptyOrUnderutilized
  consolidateAfter: 1m
limits:
  cpu: "4"
  memory: "8Gi"
```

File: infra\scale\kustomization.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - hpa.yaml
  - karpenter.yaml
  - pdb.yaml
```

File: infra\scale\pdb.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shipzen-controller-pdb
  namespace: shipzen-system
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: shipzen-controller
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shipzen-worker-pdb
  namespace: shipzen-system
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: shipzen-worker
```

File: infra\system\alertmanager-config.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: monitoring.coreos.com/v1alpha1
kind: AlertmanagerConfig
metadata:
  name: platform-routing
  namespace: shipzen-system
spec:
  route:
    receiver: slack-general
    groupBy: ['alertname', 'namespace']
    groupWait: 30s
    groupInterval: 5m
    repeatInterval: 1h
    routes:
      - matchers:
          - name: severity
            value: critical
            matchType: "="
        receiver: pagerduty-critical
        repeatInterval: 1h
      - matchers:
          - name: severity
            value: high
            matchType: "="
        receiver: slack-alerts
        repeatInterval: 5m
      - matchers:
          - name: severity
            value: warning
            matchType: "="
        receiver: slack-alerts
        repeatInterval: 24h
```

```

receivers:
  - name: slack-general
    slackConfigs:
      - apiURL:
          name: shipzen-alerts-secrets
          key: slack-webhook-url
          channel: '#shipzen-general'
          sendResolved: true
          title: '[{{ .Status | toUpper }}] {{ .GroupLabels.SortedPairs.Values | join " " }}'
          text: "{{ range .Alerts }}{{ .Annotations.description }}\n{{ end }}"

  - name: slack-alerts
    slackConfigs:
      - apiURL:
          name: shipzen-alerts-secrets
          key: slack-webhook-url
          channel: '#shipzen-alerts'
          sendResolved: true
          title: '[{{ .Status | toUpper }}] {{ .GroupLabels.SortedPairs.Values | join " " }}'
          text: "{{ range .Alerts }}{{ .Annotations.description }}\n{{ end }}"

  - name: pagerduty-critical
    pagerdutyConfigs:
      - routingKey:
          name: shipzen-alerts-secrets
          key: pagerduty-routing-key
          sendResolved: true

---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: shipzen-alerts-secrets
  namespace: shipzen-system
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: shipzen-alerts-secrets
  data:
    - secretKey: slack-webhook-url
      remoteRef:

```

```
    key: shipzen/alerts
    property: slack-webhook-url
- secretKey: pagerduty-routing-key
remoteRef:
    key: shipzen/alerts
    property: pagerduty-routing-key
```

File: infra\system\build-namespace-exception.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kyverno.io/v2
kind: PolicyException
metadata:
  name: shipzen-build-exception
  namespace: shipzen-build
spec:
  exceptions:
    - policyName: "*"
      ruleNames:
        - "*"
  match:
    any:
      - resources:
          kinds:
            - Pod
            - Job
          namespaces:
            - shipzen-build
```


File: infra\system\build-namespace.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: Namespace
metadata:
  name: shipzen-build
  labels:
    shipzen.jeneeldumasia.codes/system: "true"
    # Build pods require elevated permissions:
    # - buildkit rootless needs seccompProfile=Unconfined + apparmor unconfined
    # - BuildpackBuilder pack/dind needs privileged=true
    # "privileged" PSS level allows all of the above.
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: privileged
    pod-security.kubernetes.io/warn-version: latest
```

File: infra\system\ecr-token-rotator.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: ecr-token-rotator
  namespace: shipzen-system
spec:
  schedule: "0 */6 * * *"
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: ecr-token-rotator-sa
          containers:
            - name: rotator
              image: amazon/aws-cli:latest
              command:
                - /bin/sh
                - -c
                - |
                  echo "Fetching new ECR token..."
                  TOKEN=$(aws ecr get-login-password --region ${AWS_REGION})
                  if [ -z "$TOKEN" ]; then
                    echo "Failed to get ECR token"
                    exit 1
                  fi
                  echo "Updating Secret in AWS Secrets Manager..."
                  aws secretsmanager put-secret-value \
                    --secret-id "shipzen/ecr-pull-token" \
                    --secret-string "$TOKEN" \
```

```
    --region ${AWS_REGION}
    echo "Success!"
env:
  - name: AWS_REGION
    value: "us-east-1"
restartPolicy: OnFailure
```

File: infra\system\gateway.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: gateway.envoyproxy.io/v1alpha1
kind: EnvoyProxy
metadata:
  name: aws-nlb-proxy
  namespace: shipzen-system
spec:
  provider:
    type: Kubernetes
    kubernetes:
      envoyService:
        type: LoadBalancer
      annotations:
        service.beta.kubernetes.io/aws-load-balancer-type: "external"
        service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: "ip"
        service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
---
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: envoy-gateway
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
  parametersRef:
    group: gateway.envoyproxy.io
    kind: EnvoyProxy
    name: aws-nlb-proxy
    namespace: shipzen-system
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
```

```

name: shipzen-gateway
namespace: shipzen-system
# Annotations moved to EnvoyProxy above

spec:
  gatewayClassName: envoy-gateway
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
          from: All
      # Fix #15: redirect all HTTP traffic to HTTPS.
      # Envoy Gateway implements this via an HTTPRoute filter attached to the
      # listener. The HTTPRoute below (shipzen-http-redirect) handles the
      # actual redirect; this listener just needs to accept routes from all
      # namespaces so the redirect route can bind to it.
    - name: https
      protocol: HTTPS
      port: 443
      tls:
        mode: Terminate
        certificateRefs:
          - name: shipzen-tls-cert
      allowedRoutes:
        namespaces:
          from: All
  ---
  # Fix #15: catch-all HTTPRoute on port 80 that issues a 301 redirect to HTTPS.
  # Attach this to the http listener so all tenant traffic is upgraded.
  apiVersion: gateway.networking.k8s.io/v1
  kind: HTTPRoute
  metadata:
    name: shipzen-http-redirect
    namespace: shipzen-system
  spec:
    parentRefs:
      - name: shipzen-gateway
        namespace: shipzen-system
        sectionName: http
    hostnames:
      - "*.jeneeldumasia.codes"
      - "shipzen.jeneeldumasia.codes"
    rules:

```

```
- filters:  
  - type: RequestRedirect  
    requestRedirect:  
      scheme: https  
      statusCode: 301
```

File: infra\system\grafana-dashboards.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-dashboards
  namespace: observability
  labels:
    grafana_dashboard: "1"
data:
  platform-health.json: |-
    {
      "uid": "platform-health",
      "title": "Platform Health",
      "panels": [
        {"title": "Queue Depth", "type": "timeseries", "gridPos": {"h": 8, "w": 12, "x": 0, "y": 0},
        {"title": "DLQ Depth", "type": "timeseries", "gridPos": {"h": 8, "w": 12, "x": 12, "y": 0},
        {"title": "Retry Rate (5m)", "type": "timeseries", "gridPos": {"h": 8, "w": 8, "x": 0, "y": 8},
        {"title": "Reconciliation Duration", "type": "timeseries", "gridPos": {"h": 8, "w": 8, "x": 8, "y": 8},
        {"title": "Drift Events (5m)", "type": "timeseries", "gridPos": {"h": 8, "w": 8, "x": 16, "y": 8},
      ]
    }
  build-performance.json: |-
    {
      "apiVersion": "dashboard.grafana.app/v2",
      "kind": "Dashboard",
      "metadata": {
        "name": "build-performance",
        "namespace": "default",
        "uid": "b901c294-b684-4a3c-8820-56711b4a808a"
      },
      "spec": {
        "editable": true,
```

```

"elements": {
  "panel-1": {
    "kind": "Panel",
    "spec": {
      "data": {
        "kind": "QueryGroup",
        "spec": {
          "queries": [
            {
              "kind": "PanelQuery",
              "spec": {
                "hidden": false,
                "query": {
                  "datasource": {
                    "name": "prometheus"
                  },
                  "group": "prometheus",
                  "kind": "DataQuery",
                  "spec": {
                    "expr": "job:build_duration_seconds:p95:1h"
                  },
                  "version": "v0"
                },
                "refId": "A"
              }
            }
          ]
        }
      },
      "title": "Build Duration p95",
      "vizConfig": {
        "group": "stat",
        "kind": "VizConfig",
        "spec": {
          "options": {
            "colorMode": "value",
            "graphMode": "area",
            "justifyMode": "auto",
            "orientation": "auto",
            "reduceOptions": {
              "calcs": ["lastNotNull"],
              "fields": "",
              "values": false
            },
            "textMode": "auto",

```



```

        "wideLayout": true
      }
    },
    "version": "13.1.0"
  }
},
"panel-2": {
  "kind": "Panel",
  "spec": {
    "data": {
      "kind": "QueryGroup",
      "spec": {
        "queries": [
          {
            "kind": "PanelQuery",
            "spec": {
              "hidden": false,
              "query": {
                "datasource": {
                  "name": "prometheus"
                },
                "group": "prometheus",
                "kind": "DataQuery",
                "spec": {
                  "expr": "histogram_quantile(0.50, sum by(le) (rate(shipzen_bu
                ),
                "version": "v0"
              },
              "refId": "A"
            }
          }
        ]
      }
    },
    "title": "Build Duration p50",
    "vizConfig": {
      "group": "stat",
      "kind": "VizConfig",
      "spec": {
        "options": {
          "colorMode": "value",
          "graphMode": "area",
          "justifyMode": "auto",
          "orientation": "auto",

```

```

        "reduceOptions": {
          "calcs": ["lastNotNull"],
          "fields": "",
          "values": false
        },
        "textMode": "auto",
        "wideLayout": true
      }
    },
    "version": "13.1.0"
  }
},
"panel-3": {
  "kind": "Panel",
  "spec": {
    "data": {
      "kind": "QueryGroup",
      "spec": {
        "queries": [
          {
            "kind": "PanelQuery",
            "spec": {
              "hidden": false,
              "query": {
                "datasource": {
                  "name": "prometheus"
                },
                "group": "prometheus",
                "kind": "DataQuery",
                "spec": {
                  "expr": "rate(shipzen_deployment_success_total[1h])"
                },
                "version": "v0"
              },
              "refId": "A"
            }
          }
        ]
      }
    },
    "title": "Build Success Rate",
    "vizConfig": {
      "group": "timeseries",
      "kind": "VizConfig",

```

```

        "spec": {
          "options": {
            "legend": {
              "displayMode": "list",
              "placement": "bottom",
              "showLegend": true
            },
            "tooltip": {
              "mode": "single"
            }
          }
        },
        "version": "13.1.0"
      }
    },
    "panel-4": {
      "kind": "Panel",
      "spec": {
        "data": {
          "kind": "QueryGroup",
          "spec": {
            "queries": [
              {
                "kind": "PanelQuery",
                "spec": {
                  "hidden": false,
                  "query": {
                    "datasource": {
                      "name": "prometheus"
                    },
                    "group": "prometheus",
                    "kind": "DataQuery",
                    "spec": {
                      "expr": "rate(shipzen_deployment_failure_total[1h])"
                    },
                    "version": "v0"
                  },
                  "refId": "A"
                }
              }
            ]
          }
        },
        "title": "Build Failure Rate",

```

```

    "vizConfig": {
      "group": "timeseries",
      "kind": "VizConfig",
      "spec": {
        "options": {
          "legend": {
            "displayMode": "list",
            "placement": "bottom",
            "showLegend": true
          },
          "tooltip": {
            "mode": "single"
          }
        }
      },
      "version": "13.1.0"
    }
  },
  "panel-5": {
    "kind": "Panel",
    "spec": {
      "data": {
        "kind": "QueryGroup",
        "spec": {
          "queries": [
            {
              "kind": "PanelQuery",
              "spec": {
                "hidden": false,
                "query": {
                  "datasource": {
                    "name": "prometheus"
                  },
                  "group": "prometheus",
                  "kind": "DataQuery",
                  "spec": {
                    "expr": "kube_deployment_status_replicas_ready{deployment=\"s"
                  },
                  "version": "v0"
                },
                "refId": "A"
              }
            }
          ]
        }
      }
    }
  }
]

```

```

    }
  },
  "title": "Active Builder Pods",
  "vizConfig": {
    "group": "stat",
    "kind": "VizConfig",
    "spec": {
      "options": {
        "colorMode": "value",
        "graphMode": "area",
        "justifyMode": "auto",
        "orientation": "auto",
        "reduceOptions": {
          "calcs": ["lastNotNull"],
          "fields": "",
          "values": false
        },
        "textMode": "auto",
        "wideLayout": true
      }
    },
    "version": "13.1.0"
  }
}
},
"layout": {
  "kind": "GridLayout",
  "spec": {
    "items": [
      {"kind": "GridLayoutItem", "spec": {"element": {"kind": "ElementReference"},
      {"kind": "GridLayoutItem", "spec": {"element": {"kind": "ElementReference"},
      {"kind": "GridLayoutItem", "spec": {"element": {"kind": "ElementReference"},
      {"kind": "GridLayoutItem", "spec": {"element": {"kind": "ElementReference"},
      {"kind": "GridLayoutItem", "spec": {"element": {"kind": "ElementReference"},
    ]
  }
},
"timeSettings": {
  "from": "now-6h",
  "to": "now"
},
"title": "Build Performance"
}
}

```

```

project-resources.json: |-
{
  "uid": "project-resources",
  "title": "Per-Project Resource Usage",
  "panels": [
    {"title": "CPU Usage by Namespace", "type": "timeseries", "gridPos": {"h": 9, "w": 12, "x": 0, "y": 0},
    {"title": "Memory by Namespace", "type": "timeseries", "gridPos": {"h": 9, "w": 12, "x": 12, "y": 0},
    {"title": "Quota Saturation", "type": "timeseries", "gridPos": {"h": 8, "w": 24, "x": 0, "y": 9},
  ]
}
pod-health.json: |-
{
  "uid": "pod-health",
  "title": "Per-Deployment Pod Health",
  "templating": {
    "list": [
      {"name": "namespace", "type": "query", "query": "label_values(kube_pod_info, namespace)", "refresh": 1},
      {"name": "deployment", "type": "query", "query": "label_values(kube_pod_info{namespace=\"$namespace\"}, deployment)", "refresh": 1},
    ],
  },
  "panels": [
    {"title": "Pod Ready Status", "type": "stat", "gridPos": {"h": 6, "w": 6, "x": 0, "y": 0},
    {"title": "Restart Count", "type": "timeseries", "gridPos": {"h": 6, "w": 18, "x": 6, "y": 0},
    {"title": "CPU Usage", "type": "timeseries", "gridPos": {"h": 9, "w": 12, "x": 0, "y": 6},
    {"title": "Memory Usage", "type": "timeseries", "gridPos": {"h": 9, "w": 12, "x": 12, "y": 6},
  ]
}

```

File: infra\system\grafana-route.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: grafana-route
  namespace: observability
spec:
  parentRefs:
    - name: shipzen-gateway
      namespace: shipzen-system
  hostnames:
    - "grafana-shipzen.jeneeldumasia.codes"
  rules:
    - backendRefs:
        - name: kube-prometheus-stack-grafana
          port: 80
```

File: infra\system\kustomization.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - namespace.yaml
  - build-namespace.yaml
  - schema-configmap.yaml
  - schema-job.yaml
  - servicemonitors.yaml
  - alertmanager-config.yaml
  - shipzen-tls-cert.yaml
  - gateway.yaml
  - grafana-route.yaml
  - kyverno-exception.yaml
  - build-namespace-exception.yaml
  - grafana-dashboards.yaml
  - ecr-token-rotator.yaml
```


File: infra\system\kyverno-exception.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kyverno.io/v2
kind: PolicyException
metadata:
  name: node-exporter-exception
  namespace: observability
spec:
  exceptions:
    - policyName: "*"
      ruleNames:
        - "*"
  match:
    any:
      - resources:
          kinds:
            - Pod
            - DaemonSet
          namespaces:
            - observability
          names:
            - "*node-exporter*"
```

File: `infra\system\namespace.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\infra` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: Namespace
metadata:
  name: shipzen-system
  labels:
    shipzen.jeneeldumasia.codes/system: "true"
    # Allow platform pods to run without the restricted PSS
    # (worker, controller, and API server are trusted platform components)
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: latest
```

File: infra\system\schema-configmap.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: shipzen-schema
  namespace: shipzen-system
data:
  schema.sql: |
    -- Phase 7: PostgreSQL Database Schema

    CREATE EXTENSION IF NOT EXISTS "uuid-oss";

    -- Projects Table
    CREATE TABLE IF NOT EXISTS projects (
      id VARCHAR(255) PRIMARY KEY,
      name VARCHAR(255) NOT NULL,
      namespace VARCHAR(255) NOT NULL,
      status VARCHAR(50) NOT NULL DEFAULT 'Provisioning',
      created_at TIMESTAMP NOT NULL DEFAULT NOW(),
      deleted_at TIMESTAMP
    );

    CREATE INDEX IF NOT EXISTS idx_projects_status ON projects(status);
    CREATE INDEX IF NOT EXISTS idx_projects_created_at ON projects(created_at DESC);

    -- Deployments Table
    CREATE TABLE IF NOT EXISTS deployments (
      deployment_id VARCHAR(255) PRIMARY KEY,
      project_id VARCHAR(255) NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
      repo_url TEXT NOT NULL,
      image_uri TEXT,
      replicas INT DEFAULT 1,
```

```

        port INT DEFAULT 8080,
        state VARCHAR(50) NOT NULL,
        updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
        last_error TEXT
    );

CREATE INDEX IF NOT EXISTS idx_deployments_project_id ON deployments(project_id);
CREATE INDEX IF NOT EXISTS idx_deployments_state ON deployments(state);
CREATE INDEX IF NOT EXISTS idx_deployments_updated_at ON deployments(updated_at DESC)

-- Builds Table
CREATE TABLE IF NOT EXISTS builds (
    build_id VARCHAR(255) PRIMARY KEY,
    deployment_id VARCHAR(255) NOT NULL REFERENCES deployments(deployment_id) ON DELETE CASCADE,
    s3_log_uri TEXT,
    status VARCHAR(50) NOT NULL,
    started_at TIMESTAMP NOT NULL DEFAULT NOW(),
    completed_at TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_builds_deployment_id ON builds(deployment_id);
CREATE INDEX IF NOT EXISTS idx_builds_status ON builds(status);
CREATE INDEX IF NOT EXISTS idx_builds_started_at ON builds(started_at DESC);

-- Audit Logs (Append-Only)
CREATE TABLE IF NOT EXISTS audit_logs (
    id SERIAL PRIMARY KEY,
    project_id VARCHAR(255) REFERENCES projects(id) ON DELETE CASCADE,
    user_id VARCHAR(255) NOT NULL,
    action VARCHAR(255) NOT NULL,
    resource_type VARCHAR(50) NOT NULL,
    resource_id VARCHAR(255) NOT NULL,
    details JSONB,
    timestamp TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_audit_logs_project_id ON audit_logs(project_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_timestamp ON audit_logs(timestamp DESC);

```

File: infra\system\schema-job.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: shipzen-schema-bootstrap
  namespace: shipzen-system
  annotations:
    # ArgoCD: run this Job on every sync but treat it as a hook, not a
    # persistent resource. PostSync ensures it runs after the namespace
    # and PostgreSQL are ready.
    argocd.argoproj.io/hook: PostSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation
spec:
  # Retry up to 3 times if psql exits non-zero (e.g. postgres not ready yet)
  backoffLimit: 3
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: schema-apply
          image: postgres:15-alpine
          command:
            - sh
            - -c
            - |
              echo "Applying schema to $DATABASE_URL..."
              psql "$DATABASE_URL" -f /schema/schema.sql
              echo "Schema applied successfully."
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
```

```
        name: shipzen-db-credentials
        key: url
    volumeMounts:
      - name: schema
        mountPath: /schema
        readOnly: true
    volumes:
      - name: schema
        configMap:
          name: shipzen-schema
```

File: infra\system\servicemonitors.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
# Task 9 / fix #6.4: ServiceMonitor and headless Service resources so
# Prometheus can scrape worker, controller, and API server metrics.
# Previously no ServiceMonitors existed – none of the custom metrics
# (queue depth, drift total, reconciliation duration) were ever scraped.

# — Worker metrics service —————
apiVersion: v1
kind: Service
metadata:
  name: shipzen-worker-metrics
  namespace: shipzen-system
  labels:
    app: shipzen-worker
    shipzen.jeneeldumasia.codes/metrics: "true"
spec:
  type: ClusterIP
  clusterIP: None # Headless – Prometheus connects directly to pod IPs
  selector:
    app: shipzen-worker
  ports:
    - name: metrics
      port: 8000
      targetPort: 8000
      protocol: TCP
---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: shipzen-worker
  namespace: shipzen-system
  labels:
```

```

# kube-prometheus-stack discovers ServiceMonitors by this label
release: kube-prometheus-stack
spec:
  selector:
    matchLabels:
      shipzen.jeneeldumasia.codes/metrics: "true"
      app: shipzen-worker
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
---
# — Controller metrics service
apiVersion: v1
kind: Service
metadata:
  name: shipzen-controller-metrics
  namespace: shipzen-system
  labels:
    app: shipzen-controller
    shipzen.jeneeldumasia.codes/metrics: "true"
spec:
  type: ClusterIP
  clusterIP: None
  selector:
    app: shipzen-controller
  ports:
    - name: metrics
      port: 9090
      targetPort: 9090
      protocol: TCP
---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: shipzen-controller
  namespace: shipzen-system
  labels:
    release: kube-prometheus-stack
spec:
  selector:
    matchLabels:
      shipzen.jeneeldumasia.codes/metrics: "true"
      app: shipzen-controller
  endpoints:

```



```

    - port: metrics
      interval: 30s
      path: /metrics
---
# — API server metrics service
apiVersion: v1
kind: Service
metadata:
  name: shipzen-api-metrics
  namespace: shipzen-system
  labels:
    app: shipzen-api
    shipzen.jeneeldumasia.codes/metrics: "true"
spec:
  type: ClusterIP
  clusterIP: None
  selector:
    app: shipzen-api
  ports:
    - name: metrics
      port: 8000
      targetPort: 8000
      protocol: TCP
---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: shipzen-api
  namespace: shipzen-system
  labels:
    release: kube-prometheus-stack
spec:
  selector:
    matchLabels:
      shipzen.jeneeldumasia.codes/metrics: "true"
      app: shipzen-api
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics

```

File: infra\system\shipzen-tls-cert.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: shipzen-tls-cert
  namespace: shipzen-system
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: shipzen-tls-cert
    creationPolicy: Owner
    template:
      type: kubernetes.io/tls
  data:
    - secretKey: tls.crt
      remoteRef:
        key: shipzen/cloudflare-origin-cert
        property: cert
    - secretKey: tls.key
      remoteRef:
        key: shipzen/cloudflare-origin-cert
        property: key
```

File: infra\ui\deployment.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shipzen-ui
  namespace: shipzen-system
  labels:
    app: shipzen-ui
spec:
  replicas: 2
  selector:
    matchLabels:
      app: shipzen-ui
  template:
    metadata:
      labels:
        app: shipzen-ui
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: ui
          image: shipzen-ui:latest
          imagePullPolicy: Always
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
          ports:
```

```

- name: http
  containerPort: 3000
  protocol: TCP
env:
- name: NEXT_PUBLIC_API_URL
  value: "https://shipzen.jeneeldumasiasia.codes/api/v1"
- name: NEXT_PUBLIC_APP_DOMAIN
  value: "shipzen.jeneeldumasiasia.codes"
- name: AUTH_TRUST_HOST
  value: "true"
- name: AUTH_URL
  value: "https://shipzen.jeneeldumasiasia.codes"
- name: NODE_OPTIONS
  value: "--dns-result-order=ipv4first"
- name: AUTH_SECRET
  value: "5f8a0b1c2d3e4f5g6h7i8j9k0l1m2n3o4p5q6r7s8t9u0v1w2x3y4z5"
- name: GITHUB_CLIENT_ID
  valueFrom:
    secretKeyRef:
      name: shipzen-github
      key: client_id
      optional: true
- name: GITHUB_CLIENT_SECRET
  valueFrom:
    secretKeyRef:
      name: shipzen-github
      key: client_secret
      optional: true
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
livenessProbe:
  httpGet:
    path: /
    port: 3000
  initialDelaySeconds: 10
  periodSeconds: 15
readinessProbe:
  httpGet:
    path: /
    port: 3000

```

```
initialDelaySeconds: 5  
periodSeconds: 10
```

File: infra\ui\httproute.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shipzen-ui
  namespace: shipzen-system
spec:
  parentRefs:
    - name: shipzen-gateway
      namespace: shipzen-system
      sectionName: https
  hostnames:
    - "shipzen.jeneeldumasia.codes"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: shipzen-ui
          port: 80
```

File: `infra\ui\kustomization.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\infra` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml
  - httproute.yaml
```

File: infra\ui\service.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: Service
metadata:
  name: shipzen-ui
  namespace: shipzen-system
  labels:
    app: shipzen-ui
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 3000
      protocol: TCP
      name: http
  selector:
    app: shipzen-ui
```


File: infra\worker\deployment.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shipzen-worker
  namespace: shipzen-system
  labels:
    app: shipzen-worker
spec:
  replicas: 2
  selector:
    matchLabels:
      app: shipzen-worker
  template:
    metadata:
      labels:
        app: shipzen-worker
    spec:
      serviceAccountName: shipzen-worker-sa
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: worker
          image: shipzen-worker:latest
          imagePullPolicy: Always
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
```

```

ports:
  - name: metrics
    containerPort: 8000
    protocol: TCP
env:
  - name: AWS_ROLE_ARN
    value: "arn:aws:iam::952994886652:role/ShipZenBuilderRole"
  - name: AWS_WEB_IDENTITY_TOKEN_FILE
    value: "/var/run/secrets/eks.amazonaws.com/serviceaccount/token"
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: shipzen-db-credentials
        key: url
  - name: REDIS_HOST
    value: "redis-master.shipzen-system.svc.cluster.local"
  - name: STREAM_NAME
    value: "deploy_stream"
  - name: CONSUMER_GROUP
    value: "worker_group"
  - name: BUILDER_QUEUE_NAME
    value: "builder_queue"
  - name: S3_LOG_BUCKET
    valueFrom:
      secretKeyRef:
        name: shipzen-s3-config
        key: bucket_name
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
livenessProbe:
  httpGet:
    path: /
    port: 8000
  initialDelaySeconds: 15
  periodSeconds: 20
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /
    port: 8000

```

```
    initialDelaySeconds: 10
    periodSeconds: 10
  volumeMounts:
    - mountPath: /var/run/secrets/eks.amazonaws.com/serviceaccount
      name: aws-iam-token
      readOnly: true
  volumes:
    - name: aws-iam-token
      projected:
        defaultMode: 420
        sources:
          - serviceAccountToken:
              audience: sts.amazonaws.com
              expirationSeconds: 86400
              path: token
```

File: infra\worker\kustomization.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - serviceaccount.yaml
  - deployment.yaml
  - rbac.yaml
```

File: infra\worker\rbac.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: shipzen-worker-role
  namespace: shipzen-build
rules:
- apiGroups: ["batch"]
  resources: ["jobs"]
  verbs: ["create", "get", "delete", "list"]
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: shipzen-worker-rolebinding
  namespace: shipzen-build
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: shipzen-worker-role
subjects:
- kind: ServiceAccount
  name: shipzen-worker-sa
  namespace: shipzen-system
```

File: infra\worker\serviceaccount.yaml

Purpose & Responsibility: This file is a critical piece of the C:\Project\ShipZen\infra subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: shipzen-worker-sa
  namespace: shipzen-system
  annotations:
    eks.amazonaws.com/role-arn: "arn:aws:iam::952994886652:role/ShipZenBuilderRole"
automountServiceAccountToken: true # Worker now orchestrates Builder jobs in K8s
```

7. CI/CD Pipeline Deep Dive

File: `.github\workflows\auto-destroy.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Auto-Destroy (Deadman Switch)

# Runs every 8 hours. If the EKS cluster has been alive for more than 8 hours,
# it triggers the destroy workflow automatically to protect credits.
# Adjust MAX_UPTIME_HOURS to your preference.

on:
  schedule:
    - cron: '0 * * * *' # Every hour
  workflow_dispatch:      # Also allow manual trigger for testing

permissions:
  id-token: write
  contents: read
  actions: write # Required to trigger the destroy workflow

env:
  MAX_UPTIME_HOURS: 6
  CLUSTER_NAME: shipzen-cluster
  AWS_REGION: ${vars.AWS_REGION || 'us-east-1' }}

jobs:
  check-and-destroy:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS Credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
```

```

    role-to-assume: ${ secrets.AWS_ROLE_ARN }}
    aws-region: ${ env.AWS_REGION }}

- name: Check Cluster Uptime
  id: check
  run: |
    # Check if the cluster exists at all
    CREATED_AT=$(aws eks describe-cluster \
      --name "$CLUSTER_NAME" \
      --query "cluster.createdAt" \
      --output text 2>/dev/null || echo "")

    VPC_ID=$(aws ec2 describe-vpcs --filters "Name=tag:Name,Values=shipzen-vpc" --q

    if [ "$VPC_ID" != "None" ] && ([ -z "$CREATED_AT" ] || [ "$CREATED_AT" = "None"
      echo "⚠️ Cluster is missing but VPC ($VPC_ID) still exists! This means a part
      echo "Triggering auto-destroy to clean up lingering resources."
      echo "should_destroy=true" >> $GITHUB_OUTPUT
      echo "uptime_hours=0" >> $GITHUB_OUTPUT
      exit 0
    fi

    if [ -z "$CREATED_AT" ] || [ "$CREATED_AT" = "None" ]; then
      echo "Cluster and VPC not found. Nothing to destroy."
      echo "should_destroy=false" >> $GITHUB_OUTPUT
      exit 0
    fi

    # Calculate uptime in hours
    CREATED_EPOCH=$(date -d "$CREATED_AT" +%s)
    NOW_EPOCH=$(date +%s)
    UPTIME_SECONDS=$((NOW_EPOCH - CREATED_EPOCH))
    UPTIME_HOURS=$(echo "scale=2; $UPTIME_SECONDS / 3600" | bc)

    echo "Cluster created at: $CREATED_AT"
    echo "Uptime: ${UPTIME_HOURS} hours (limit: ${MAX_UPTIME_HOURS}h)"

    if [ "$UPTIME_SECONDS" -gt $((MAX_UPTIME_HOURS * 3600)) ]; then
      echo "⚠️ Cluster has exceeded ${MAX_UPTIME_HOURS}h uptime. Triggering auto-c
      echo "should_destroy=true" >> $GITHUB_OUTPUT
      echo "uptime_hours=$UPTIME_HOURS" >> $GITHUB_OUTPUT
    else
      echo "✅ Cluster within allowed uptime. No action needed."
      echo "should_destroy=false" >> $GITHUB_OUTPUT
    fi
  fi

```



```
- name: Trigger Destroy Workflow
  if: steps.check.outputs.should_destroy == 'true'
  uses: actions/github-script@v7
  with:
    script: |
      await github.rest.actions.createWorkflowDispatch({
        owner: context.repo.owner,
        repo: context.repo.repo,
        workflow_id: 'destroy.yaml',
        ref: 'main'
      });
      console.log(`Auto-destroy triggered. Cluster uptime was ${steps.check.outpu
```

File: `.github\workflows\build-push.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Build and Push Docker Images
on:
  push:
    branches:
      - main
    paths:
      - 'api/**'
      - 'ui/**'
      - 'controller/**'
      - 'worker/**'
      - '.github/workflows/build-push.yaml'
  workflow_dispatch:

env:
  REGISTRY: ghcr.io

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      packages: write
    strategy:
      matrix:
        component: [api, ui, controller, worker]
    outputs:
      api_tag:      ${ steps.tag.outputs.tag }
      ui_tag:       ${ steps.tag.outputs.tag }
      controller_tag: ${ steps.tag.outputs.tag }
      worker_tag:   ${ steps.tag.outputs.tag }
    steps:
```

```

- name: Checkout repository
  uses: actions/checkout@v4

- name: Convert repository owner to lowercase
  id: string
  uses: ASzc/change-string-case-action@v6
  with:
    string: ${{ github.repository_owner }}

- name: Set image tag
  id: tag
  run: echo "tag=sha-$(git rev-parse --short HEAD)" >> "$GITHUB_OUTPUT"

- name: Log in to the Container registry
  uses: docker/login-action@v3
  with:
    registry: ${{ env.REGISTRY }}
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Extract metadata (tags, labels) for Docker
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${{ env.REGISTRY }}/${{ steps.string.outputs.lowercase }}/shipzen-${{ m
    tags: |
      type=sha,format=short
      type=raw,value=latest

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: ./${{ matrix.component }}
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    build-args: |
      NEXT_PUBLIC_API_URL=https://shipzen.jeneeldumasia.codes/api/v1

```

update-kustomization:

```

# Single job – runs after ALL matrix builds succeed.
# Updating kustomization.yaml in one atomic commit avoids the race condition
# where parallel matrix jobs clobber each other's tag updates.
needs: build-and-push
runs-on: ubuntu-latest

```

```

permissions:
  contents: write
steps:
  - name: Checkout repository
    uses: actions/checkout@v4
    with:
      # Fetch the very latest commit so we rebase on top of it
      ref: main
      fetch-depth: 0

  - name: Update all component tags in kustomization.yaml
    run: |
      SHORT_SHA=$(git rev-parse --short HEAD)
      TAG="sha-${SHORT_SHA}"
      echo "Updating all built components to tag: $TAG"

      python3 - <<'PYEOF'
      import re, os, sys

      tag = os.environ["TAG"]
      components = ["api", "ui", "controller", "worker"]
      path = "infra/kustomization.yaml"

      with open(path, "r") as f:
          lines = f.readlines()

      for i, line in enumerate(lines):
          for c in components:
              if f"name: shipzen-{c}" in line:
                  # newTag is always 2 lines below the name: line
                  lines[i + 2] = re.sub(r"newTag: .*", f"newTag: {tag}\n", lines[i +
      with open(path, "w") as f:
          f.writelines(lines)
      PYEOF
    env:
      TAG: sha-$(git rev-parse --short HEAD)

  - name: Commit and push tag update
    run: |
      SHORT_SHA=$(git rev-parse --short HEAD)
      TAG="sha-${SHORT_SHA}"

      git config user.name "GitHub Actions Bot"
      git config user.email "actions@github.com"
      git add infra/kustomization.yaml

```

```
if git diff-index --quiet HEAD; then
    echo "No tag changes to commit."
    exit 0
fi

git commit -m "chore: update platform images to ${TAG} [skip ci]"

# Retry loop handles any last-second push from another workflow
for i in {1..5}; do
    git pull --rebase origin main
    if git push origin main; then
        echo "Successfully pushed kustomization update"
        exit 0
    fi
    echo "Push attempt $i failed, retrying in 5s..."
    sleep 5
done
echo "ERROR: failed to push after 5 attempts"
exit 1
```

File: `.github\workflows\debug.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Debug Cluster
on:
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

jobs:
  debug:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          aws-region: ${ vars.AWS_REGION || 'us-east-1' }

      - name: Debug Cluster
        run: |
          aws eks update-kubeconfig --region ${ vars.AWS_REGION || 'us-east-1' } --name
          echo "=== NODES ==="
          kubectl get nodes -o wide || true
          echo "=== PODS ==="
          kubectl get pods -A -o wide || true
          echo "=== EVENTS ==="
          kubectl get events -A --sort-by='.lastTimestamp' | tail -n 50 || true
          echo "=== WEBHOOKS ==="
          kubectl get validatingwebhookconfigurations || true
```

```
kubect1 get mutatingwebhookconfigurations || true
```



File: `.github\workflows\deploy-secrets.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Deploy Secrets to EKS

on:
  push:
    branches:
      - main
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

jobs:
  deploy-secrets:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::952994886652:role/ShipZenGitHubActionsRole
          aws-region: us-east-1

      - name: Update kubeconfig
        run: aws eks update-kubeconfig --region us-east-1 --name shipzen-cluster

      - name: Deploy GitHub OAuth Secret
        run: |
          kubectl create secret generic shipzen-github \
```



```
--from-literal=enabled=true \  
--from-literal=client_id=${{ secrets.SHIPZEN_GITHUB_CLIENT_ID }} \  
--from-literal=client_secret=${{ secrets.SHIPZEN_GITHUB_CLIENT_SECRET }} \  
-n shipzen-system --dry-run=client -o yaml | kubectl apply -f -
```

File: `.github\workflows\deploy.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Deploy Platform Infra

on:
  push:
    branches:
      - main
    paths:
      - 'terraform/**'
      - '.github/workflows/deploy.yaml'
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

jobs:
  terraform-deploy:
    runs-on: ubuntu-latest
    env:
      ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION: true
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS Credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          aws-region: ${ vars.AWS_REGION || 'us-east-1' }
```

```

- name: Setup Terraform
  uses: hashicorp/setup-terraform@v3
  with:
    cli_config_credentials_token: ${ secrets.TF_API_TOKEN }

- name: Terraform Init
  working-directory: ./terraform
  run: terraform init

- name: Cleanup Stuck Webhooks
  working-directory: ./terraform
  run: |
    aws eks update-kubeconfig --region ${ vars.AWS_REGION || 'us-east-1' } --name
    kubectl delete validatingwebhookconfigurations --all || true
    kubectl delete mutatingwebhookconfigurations --all || true

# Bootstrap: Ensure the GitHub Actions OIDC role has EKS cluster admin access.
# This uses the AWS API (not the Kubernetes API), so it works even when
# the role has zero Kubernetes RBAC permissions. Idempotent – safe to run
# on every deploy. Solves the chicken-and-egg problem where Terraform's
# kubernetes/helm providers need RBAC access that only Terraform can grant.
- name: Bootstrap EKS Access for GitHub Actions Role
  run: |
    ROLE_ARN="arn:aws:iam:${ secrets.AWS_ACCOUNT_ID || '952994886652' }:role/Shi
    CLUSTER="shipzen-cluster"
    REGION="${ vars.AWS_REGION || 'us-east-1' }"

    echo "Ensuring EKS access entry exists for $ROLE_ARN..."
    aws eks create-access-entry \
      --cluster-name "$CLUSTER" \
      --principal-arn "$ROLE_ARN" \
      --type STANDARD \
      --region "$REGION" 2>/dev/null || echo "Access entry already exists."

    echo "Associating cluster admin policy..."
    aws eks associate-access-policy \
      --cluster-name "$CLUSTER" \
      --principal-arn "$ROLE_ARN" \
      --policy-arn "arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPol
      --access-scope type=cluster \
      --region "$REGION" 2>/dev/null || echo "Policy already associated."

    echo "EKS access bootstrap complete."

```

```

- name: Terraform Plan
  working-directory: ./terraform
  env:
    TF_VAR_cloudflare_api_token: ${ secrets.CLOUDFLARE_API_TOKEN }
  run: terraform plan -out=tfplan

- name: Terraform Apply
  working-directory: ./terraform
  env:
    TF_VAR_cloudflare_api_token: ${ secrets.CLOUDFLARE_API_TOKEN }
  run: terraform apply tfplan

- name: Configure kubectl
  run: aws eks update-kubeconfig --region ${ vars.AWS_REGION || 'us-east-1' } --n

- name: Force cleanup cert-manager resources
  run: |
    echo "Removing finalizers from cert-manager resources..."
    for crd in certificates.cert-manager.io clusterissuers.cert-manager.io challeng
      kubectl get $crd -A -o name 2>/dev/null | xargs -I {} kubectl patch {} -p '{"
      kubectl delete $crd --all -A --wait=false 2>/dev/null || true
    done
    echo "Triggering ArgoCD sync..."
    kubectl patch application shipzen-platform -n argocd -p '{"operation":{"sync":{"
    sleep 10

- name: Wait for Network Load Balancer
  run: |
    echo "Waiting for Network Load Balancer to be provisioned by AWS..."
    ELAPSED=0
    MAX_WAIT=600
    NLB_URL=""
    while [ $ELAPSED -lt $MAX_WAIT ]; do
      # Scan all namespaces for the LoadBalancer service instead of hardcoding the
      NLB_URL=$(kubectl get svc -A -o jsonpath="{.items[?(@.spec.type=='LoadBalance
      if [ -n "$NLB_URL" ]; then
        echo "NLB is ready: $NLB_URL"
        break
      fi
      echo "NLB not ready yet (${ELAPSED}s/${MAX_WAIT}s)..."
      sleep 10
      ELAPSED=$((ELAPSED+10))
    done

```

```

if [ -z "$NLB_URL" ]; then
    echo "Error: NLB was not provisioned in time. Dumping cluster state for debug
    kubectl get svc -A || true
    kubectl get gatewayclass -A || true
    kubectl get gateway -A -o yaml || true
    kubectl get externalsecret -A -o yaml || true
    kubectl get secret shipzen-tls-cert -n shipzen-system || true
    echo "--- DESCRIBING ARGOCD APP TO FIND SYNC ERRORS ---"
    kubectl describe application shipzen-platform -n argocd || true
    echo "--- DRY RUNNING KUSTOMIZE TO FIND VALIDATION ERRORS ---"
    kubectl apply -k infra/system --dry-run=server || true
    kubectl get events -A --sort-by='.lastTimestamp' | tail -n 50 || true
    exit 1
fi
echo "NLB_URL=$NLB_URL" >> $GITHUB_ENV

- name: Automate Cloudflare CNAME
run: |
    echo "Updating Cloudflare DNS..."
    ZONE_NAME="jeneeldumasia.codes"
    RECORDS=("*" "shipzen")

    # Get Zone ID
    ZONE_ID=$(curl -s -X GET "https://api.cloudflare.com/client/v4/zones?name=$ZONE_
        -H "Authorization: Bearer ${ secrets.CLOUDFLARE_API_TOKEN }" \
        -H "Content-Type: application/json" | jq -r '.result[0].id')

    if [ -z "$ZONE_ID" ] || [ "$ZONE_ID" == "null" ]; then
        echo "Error: Could not retrieve Zone ID for $ZONE_NAME. Please check your Clo
        exit 1
    fi

    for RECORD_NAME in "${RECORDS[@]"; do
        if [ "$RECORD_NAME" == "*" ]; then
            FULL_RECORD_NAME="*.jeneeldumasia.codes"
        else
            FULL_RECORD_NAME="shipzen.jeneeldumasia.codes"
        fi

        echo "Processing record: $FULL_RECORD_NAME"

        # Check if record exists
        RECORD_ID=$(curl -s -X GET "https://api.cloudflare.com/client/v4/zones/$ZONE_
            -H "Authorization: Bearer ${ secrets.CLOUDFLARE_API_TOKEN }" \
            -H "Content-Type: application/json" | jq -r '.result[0].id')

```

```

if [ -z "$RECORD_ID" ] || [ "$RECORD_ID" == "null" ]; then
    curl -s -X POST "https://api.cloudflare.com/client/v4/zones/$ZONE_ID/dns_re
    -H "Authorization: Bearer ${ secrets.CLOUDFLARE_API_TOKEN }" \
    -H "Content-Type: application/json" \
    --data '{"type":"CNAME","name":"'$RECORD_NAME'", "content": "'${ env.
    echo "Created new CNAME record $RECORD_NAME."
else
    curl -s -X PUT "https://api.cloudflare.com/client/v4/zones/$ZONE_ID/dns_rec
    -H "Authorization: Bearer ${ secrets.CLOUDFLARE_API_TOKEN }" \
    -H "Content-Type: application/json" \
    --data '{"type":"CNAME","name":"'$RECORD_NAME'", "content": "'${ env.
    echo "Updated existing CNAME record $RECORD_NAME."
fi
done

echo "### Deployment Fully Automated :rocket:" >> $GITHUB_STEP_SUMMARY
echo "***Network Load Balancer:** \`${ env.NLB_URL }\`" >> $GITHUB_STEP_SUMMAR
echo ":white_check_mark: **DNS Record:** Automated via Cloudflare API" >> $GITH
echo ":white_check_mark: **TLS Certificate:** Cloudflare Origin CA synced via E

# Force trigger pipeline

```

File: `.github\workflows\destroy.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Infra Teardown
on:
  workflow_dispatch: # Manual trigger only to prevent accidental destruction

permissions:
  id-token: write
  contents: read

jobs:
  terraform-destroy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS Credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          aws-region: ${ vars.AWS_REGION || 'us-east-1' }

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          cli_config_credentials_token: ${ secrets.TF_API_TOKEN }

      - name: Install kubectl
        uses: azure/setup-kubectl@v3

      - name: Terraform Init
        working-directory: ./terraform
```

```

run: terraform init

- name: Configure kubectl
  run: |
    aws eks update-kubeconfig --region ${vars.AWS_REGION} --name

# Step 1: Suspend ArgoCD auto-sync FIRST.
# If we skip this, ArgoCD's selfHeal will re-create every resource
# we delete in the next steps, causing a race condition.
- name: Suspend ArgoCD Auto-Sync
  if: env.cluster_not_found != 'true'
  run: |
    kubectl patch application shipzen-platform \
      -n argocd \
      --type merge \
      -p '{"spec":{"syncPolicy":{"automated":null}}}' \
      || echo "ArgoCD application not found, continuing..."

# Step 2: Delete Karpenter NodePools and EC2NodeClass so Karpenter
# stops provisioning new nodes and begins draining its own nodes.
# These nodes are NOT managed by Terraform, so we must remove them
# explicitly – they hold ENIs that block VPC deletion.
- name: Delete Karpenter NodePools and NodeClasses
  if: env.cluster_not_found != 'true'
  run: |
    kubectl delete nodepool --all --ignore-not-found=true || true
    kubectl delete ec2nodeclass --all --ignore-not-found=true || true

# Step 3: Drain and delete any remaining Karpenter-provisioned nodes.
# We identify them by the karpenter.sh/nodepool label.
# This ensures their ENIs are released before Terraform runs.
- name: Drain Karpenter Nodes
  if: env.cluster_not_found != 'true'
  run: |
    KARPENTER_NODES=$(kubectl get nodes -l karpenter.sh/nodepool \
      -o jsonpath='{.items[*].metadata.name}' 2>/dev/null || echo "")

    if [ -n "$KARPENTER_NODES" ]; then
      for NODE in $KARPENTER_NODES; do
        echo "Draining node: $NODE"
        kubectl drain "$NODE" \
          --ignore-daemonsets \
          --delete-emptydir-data \
          --force \
          --timeout=120s || true
      done
    fi

```



```

done
kubectl delete node $KARPENTER_NODES --ignore-not-found=true || true
else
    echo "No Karpenter nodes found."
fi

# Step 3.5: Clean up Cloudflare DNS records to prevent dangling records
- name: Cleanup Cloudflare DNS
env:
    CLOUDFLARE_API_TOKEN: ${ secrets.CLOUDFLARE_API_TOKEN }
run: |
    echo "Cleaning up Cloudflare DNS records..."
    ZONE_NAME="jeneeldumasia.codes"
    RECORDS=("*.shipzen.jeneeldumasia.codes" "shipzen.jeneeldumasia.codes")

    ZONE_ID=$(curl -s -X GET "https://api.cloudflare.com/client/v4/zones?name=$ZONE_
        -H "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
        -H "Content-Type: application/json" | jq -r '.result[0].id')

    if [ -z "$ZONE_ID" ] || [ "$ZONE_ID" == "null" ]; then
        echo "Could not retrieve Zone ID, skipping DNS cleanup."
        exit 0
    fi

    for FULL_RECORD_NAME in "${RECORDS[@]}"; do
        RECORD_ID=$(curl -s -X GET "https://api.cloudflare.com/client/v4/zones/$ZONE_
            -H "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
            -H "Content-Type: application/json" | jq -r '.result[0].id')

        if [ -n "$RECORD_ID" ] && [ "$RECORD_ID" != "null" ]; then
            echo "Deleting DNS record $FULL_RECORD_NAME (ID: $RECORD_ID)"
            curl -s -X DELETE "https://api.cloudflare.com/client/v4/zones/$ZONE_ID/dns_
                -H "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
                -H "Content-Type: application/json"
        else
            echo "DNS record $FULL_RECORD_NAME not found, skipping."
        fi
    done

# Step 4: Delete HTTPRoutes and Services of type LoadBalancer FIRST,
# then wait for AWS to fully deprovision the NLB/ELB.
# This is the #1 cause of VPC deletion failures – dangling ENIs from
# load balancers that Terraform doesn't manage directly.
- name: Delete LoadBalancer Services and HTTPRoutes
  if: env.cluster_not_found != 'true'

```

```

run: |
    # Delete gateway HTTPRoutes (tenant workload routes)
    kubectl delete httproute --all --all-namespaces --ignore-not-found=true || true

    # Delete ingress resources (these provision AWS ALBs)
    kubectl delete ingress --all --all-namespaces --ignore-not-found=true || true

    # Delete only LoadBalancer-type services (these provision AWS NLBs)
    # We do NOT delete all services – ClusterIP services are fine to leave for Terraform
    kubectl get svc --all-namespaces -o json | \
        jq -r '.items[] | select(.spec.type == "LoadBalancer") | "\(.metadata.namespace) \(.metadata.name)"' | \
        while read NS NAME; do
            echo "Deleting LoadBalancer service $NS/$NAME"
            kubectl delete svc "$NAME" -n "$NS" --ignore-not-found=true || true
        done

# Step 5: Force Delete ALL Load Balancers in our VPC.
# Kubernetes auto-generates LB names (e.g. 'a96304a043ff34844875f422f978eb9a')
# that do NOT contain 'shipzen', so name-based search misses them.
# We find them by VPC ID instead – if it's in our VPC, it must go.
- name: Force Delete Load Balancers, Target Groups, ENIs, EIPs, and Security Groups
  run: |
    VPC_ID=$(aws ec2 describe-vpcs \
        --filters "Name=tag:Name,Values=shipzen-vpc" \
        --query "Vpcs[0].VpcId" \
        --output text 2>/dev/null || echo "")

    if [ -z "$VPC_ID" ] || [ "$VPC_ID" = "None" ]; then
        echo "VPC not found, skipping LB/SG cleanup."
        exit 0
    fi

    echo "Found VPC: $VPC_ID"

    # --- Pre-clean orphaned Target Groups in this VPC ---
    # Some stale TGs can block LB deletion/deregistration chains.
    echo "=== Pre-cleaning orphaned Target Groups ==="
    PRE_TG_ARNS=$(aws elbv2 describe-target-groups \
        --query "TargetGroups[?VpcId=='$VPC_ID'].TargetGroupArn" \
        --output text 2>/dev/null || echo "")

    for TG in $PRE_TG_ARNS; do
        echo "Pre-clean deleting target group: $TG"
        aws elbv2 delete-target-group --target-group-arn "$TG" 2>/dev/null || true
    done

```

```

# --- Delete Classic ELBs (v1 API) ---
# Kubernetes cloud-controller-manager creates Classic ELBs when the
# AWS Load Balancer Controller isn't ready. These are INVISIBLE to
# the elbv2 API and must be found via the v1 elb API.
echo "=== Checking for Classic ELBs (v1) ==="
CLASSIC_LBS=$(aws elb describe-load-balancers \
  --query "LoadBalancerDescriptions[?VPCId=='$VPC_ID'].LoadBalancerName" \
  --output text 2>/dev/null || echo "")

if [ -n "$CLASSIC_LBS" ]; then
  for LB_NAME in $CLASSIC_LBS; do
    echo "Deleting Classic ELB: $LB_NAME"
    aws elb delete-load-balancer --load-balancer-name "$LB_NAME" 2>/dev/null ||
  done
else
  echo "No Classic ELBs found."
fi

# --- Delete ALBs/NLBs (v2 API) ---
echo "=== Checking for ALBs/NLBs (v2) ==="
ALL_LB_ARNS=$(aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?VpcId=='$VPC_ID'].LoadBalancerArn" \
  --output text 2>/dev/null || echo "")

if [ -n "$ALL_LB_ARNS" ]; then
  for ARN in $ALL_LB_ARNS; do
    echo "Deleting load balancer: $ARN"
    aws elbv2 delete-load-balancer --load-balancer-arn "$ARN" 2>/dev/null || tr
  done
else
  echo "No ALBs/NLBs found."
fi

# --- Wait for ALL LB ENIs to be fully released ---
echo "=== Waiting for LB ENIs to be released ==="
MAX_WAIT=180
ELAPSED=0
while [ $ELAPSED -lt $MAX_WAIT ]; do
  ELB_ENI_COUNT=$(aws ec2 describe-network-interfaces \
    --filters "Name=vpc-id,Values=$VPC_ID" "Name=description,Values=ELB *" \
    --query "length(NetworkInterfaces)" \
    --output text 2>/dev/null || echo "0")

  if [ "$ELB_ENI_COUNT" = "0" ]; then

```

```

        echo "All LB ENIs released."
        break
    fi

    echo "Still $ELB_ENI_COUNT LB ENIs in VPC... (${ELAPSED}s/${MAX_WAIT}s)"
    sleep 15
    ELAPSED=$((ELAPSED + 15))
done

# --- Delete orphaned Target Groups in this VPC ---
echo "=== Cleaning up orphaned Target Groups ==="
TG_ARNS=$(aws elbv2 describe-target-groups \
    --query "TargetGroups[?VpcId=='$VPC_ID'].TargetGroupArn" \
    --output text 2>/dev/null || echo "")

for TG in $TG_ARNS; do
    echo "Deleting target group: $TG"
    aws elbv2 delete-target-group --target-group-arn "$TG" 2>/dev/null || true
done

# --- Release Elastic IPs associated with this VPC ---
# NLBs provision EIPs in public subnets. If these aren't released,
# the internet gateway can't detach and the VPC can't be deleted.
echo "=== Releasing Elastic IPs ==="
SUBNET_IDS=$(aws ec2 describe-subnets \
    --filters "Name=vpc-id,Values=$VPC_ID" \
    --query "Subnets[*].SubnetId" \
    --output text 2>/dev/null || echo "")

# Find EIPs associated with ENIs in our VPC
EIP_ALLOCS=$(aws ec2 describe-addresses \
    --query "Addresses[?Domain=='vpc'].{AllocationId:AllocationId,AssociationId:AssociationId}" \
    --output json 2>/dev/null || echo "[]")

# Get all ENIs in our VPC to cross-reference
VPC_ENI_IDS=$(aws ec2 describe-network-interfaces \
    --filters "Name=vpc-id,Values=$VPC_ID" \
    --query "NetworkInterfaces[*].NetworkInterfaceId" \
    --output text 2>/dev/null || echo "")

echo "$EIP_ALLOCS" | jq -r '[] | "\(.AllocationId) \(.AssociationId) \(.NetworkInterfaceId)"'
if echo "$VPC_ENI_IDS" | grep -qw "$ENI" 2>/dev/null; then
    if [ "$ASSOC" != "null" ] && [ -n "$ASSOC" ]; then
        echo "Disassociating EIP: $ALLOC (association: $ASSOC)"
        aws ec2 disassociate-address --association-id "$ASSOC" 2>/dev/null || true
    fi
fi

```

```

        fi
        echo "Releasing EIP: $ALLOC"
        aws ec2 release-address --allocation-id "$ALLOC" 2>/dev/null || true
    fi
done

# --- Force-detach and delete orphaned ENIs ---
echo "=== Cleaning up orphaned ENIs ==="
ENIS=$(aws ec2 describe-network-interfaces \
    --filters "Name=vpc-id,Values=$VPC_ID" \
    --query "NetworkInterfaces[?Status=='available' || contains(Description, 'ELB'"
    --output json 2>/dev/null || echo "[]")

echo "$ENIS" | jq -r '.[ ] | "\(.ID) \(.AttachId) \(.Status)"' 2>/dev/null | while read ID ATTACH_ID STATUS; do
    if [ "$STATUS" = "in-use" ] && [ "$ATTACH_ID" != "null" ] && [ -n "$ATTACH_ID" ]; then
        echo "Force-detaching ENI: $ENI_ID"
        aws ec2 detach-network-interface --attachment-id "$ATTACH_ID" --force 2>/dev/null
        sleep 5
    fi
done

echo "Deleting ENI: $ENI_ID"
aws ec2 delete-network-interface --network-interface-id "$ENI_ID" 2>/dev/null

done

# --- Delete orphaned Security Groups ---
echo "=== Cleaning up orphaned Security Groups ==="
# Wait a moment for ENI deletions to propagate
sleep 10
SG_IDS=$(aws ec2 describe-security-groups \
    --filters "Name=vpc-id,Values=$VPC_ID" \
    --query "SecurityGroups[?GroupName!='default' && !contains(GroupName, 'shipze'"
    --output text 2>/dev/null || echo "")

if [ -n "$SG_IDS" ]; then
    # First, remove all ingress/egress rules that reference other SGs (breaks cir
    for SG in $SG_IDS; do
        echo "Revoking all rules from SG: $SG"
        aws ec2 revoke-security-group-ingress --group-id "$SG" \
            --ip-permissions "$(aws ec2 describe-security-groups --group-ids "$SG" --
        aws ec2 revoke-security-group-egress --group-id "$SG" \
            --ip-permissions "$(aws ec2 describe-security-groups --group-ids "$SG" --
    done
    # Then delete them
    for SG in $SG_IDS; do
        echo "Deleting security group: $SG"
        aws ec2 delete-security-group --group-id "$SG" 2>/dev/null || \

```

```

        echo "Could not delete $SG (may still have dependencies)"
    done
else
    echo "No orphaned security groups found."
fi

# Step 8.5: Force Uninstall Problematic Helm Releases
# Kyverno often hangs during terraform destroy because its cleanup job
# fails to run or its webhooks cause a deadlock. Force uninstalling it
# without hooks ensures Terraform can cleanly remove it from state.
- name: Force Uninstall Problematic Helm Releases
  if: env.cluster_not_found != 'true'
  run: |
    echo "Force uninstalling Kyverno without hooks..."
    helm uninstall kyverno-policies -n kyverno --no-hooks --ignore-not-found || true
    helm uninstall kyverno -n kyverno --no-hooks --ignore-not-found || true
    echo "Force uninstalling KEDA without hooks..."
    helm uninstall keda -n keda --no-hooks --ignore-not-found || true
    echo "Deleting all validating and mutating webhooks to prevent deadlocks..."
    kubectl delete validatingwebhookconfigurations --all --ignore-not-found=true || true
    kubectl delete mutatingwebhookconfigurations --all --ignore-not-found=true || true

# Step 9: Full Terraform destroy. By this point:
# - Karpenster nodes are gone (ENIs released)
# - Load balancers are gone (ENIs and SGs released)
# - App namespaces are terminated
# - ArgoCD is not fighting us
# - Kyverno is forcefully uninstalled
# Terraform should be able to cleanly destroy EKS, the VPC, and S3.
- name: Cleanup Stuck Namespaces
  if: env.cluster_not_found != 'true'
  run: |
    for ns in $(kubectl get ns | grep Terminating | awk '{print $1}'); do
      echo "Force deleting stuck namespace: $ns"
      kubectl get ns "$ns" -o json | jq '.spec.finalizers=[]' | kubectl replace --raw
    done
- name: Terraform Destroy
  working-directory: ./terraform
  env:
    TF_VAR_cloudflare_api_token: ${ secrets.CLOUDFLARE_API_TOKEN }
  run: terraform destroy -auto-approve
  continue-on-error: true
  id: first_destroy

# If first destroy failed, clean up again and retry

```

```

- name: Retry Cleanup and Terraform Destroy
  if: steps.first_destroy.outcome == 'failure'
  working-directory: ./terraform
  env:
    TF_VAR_cloudflare_api_token: ${ secrets.CLOUDFLARE_API_TOKEN }
  run: |
    echo "First terraform destroy failed. Running cleanup and retrying..."

    echo "=== Sweeping for stuck namespaces from first run ==="
    for ns in $(kubectl get ns 2>/dev/null | grep Terminating | awk '{print $1}');
      echo "Force deleting stuck namespace: $ns"
      kubectl get ns "$ns" -o json | jq '.spec.finalizers=[]' | kubectl replace --r
done

VPC_ID=$(aws ec2 describe-vpcs \
  --filters "Name=tag:Name,Values=shipzen-vpc" \
  --query "Vpcs[0].VpcId" \
  --output text 2>/dev/null || echo "")

if [ -n "$VPC_ID" ] && [ "$VPC_ID" != "None" ]; then
  # Pre-clean orphaned target groups first (best-effort)
  aws elbv2 describe-target-groups \
    --query "TargetGroups[?VpcId=='$VPC_ID'].TargetGroupArn" \
    --output text 2>/dev/null | xargs -n1 aws elbv2 delete-target-group --targe

  # Kill Classic ELBs
  aws elb describe-load-balancers \
    --query "LoadBalancerDescriptions[?VPCId=='$VPC_ID'].LoadBalancerName" \
    --output text 2>/dev/null | xargs -n1 aws elb delete-load-balancer --load-b

  # Kill v2 LBs
  aws elbv2 describe-load-balancers \
    --query "LoadBalancers[?VpcId=='$VPC_ID'].LoadBalancerArn" \
    --output text 2>/dev/null | xargs -n1 aws elbv2 delete-load-balancer --load

  # Retry target group cleanup after LB deletes
  aws elbv2 describe-target-groups \
    --query "TargetGroups[?VpcId=='$VPC_ID'].TargetGroupArn" \
    --output text 2>/dev/null | xargs -n1 aws elbv2 delete-target-group --targe

  echo "Waiting 90s for AWS to release resources..."
  sleep 90

  # Force cleanup ENIs
  aws ec2 describe-network-interfaces \

```

```

--filters "Name=vpc-id,Values=$VPC_ID" \
--query "NetworkInterfaces[*].{ID:NetworkInterfaceId,AttachId:Attachment.At
--output json 2>/dev/null | jq -r '.[ ] | "\(.ID) \(.AttachId)"' | while rea
if [ "$ATTACH_ID" != "null" ] && [ -n "$ATTACH_ID" ]; then
    aws ec2 detach-network-interface --attachment-id "$ATTACH_ID" --force 2
    sleep 3
fi
    aws ec2 delete-network-interface --network-interface-id "$ENI_ID" 2>/dev/
done

# Release all EIPs in VPC
VPC_ENI_IDS=$(aws ec2 describe-network-interfaces \
--filters "Name=vpc-id,Values=$VPC_ID" \
--query "NetworkInterfaces[*].NetworkInterfaceId" \
--output text 2>/dev/null || echo "")
aws ec2 describe-addresses --query "Addresses[?Domain=='vpc']" --output json
jq -r '.[ ].AllocationId' | while read ALLOC; do
    aws ec2 release-address --allocation-id "$ALLOC" 2>/dev/null || true
done

# Delete SGs
SG_IDS=$(aws ec2 describe-security-groups \
--filters "Name=vpc-id,Values=$VPC_ID" \
--query "SecurityGroups[?GroupName!='default' && !contains(GroupName, 'ship
--output text 2>/dev/null || echo "")
for SG in $SG_IDS; do
    aws ec2 revoke-security-group-ingress --group-id "$SG" \
        --ip-permissions "$(aws ec2 describe-security-groups --group-ids "$SG" --
    aws ec2 revoke-security-group-egress --group-id "$SG" \
        --ip-permissions "$(aws ec2 describe-security-groups --group-ids "$SG" --
    aws ec2 delete-security-group --group-id "$SG" 2>/dev/null || true
done
fi

echo "Retrying terraform destroy..."
terraform destroy -auto-approve

# Step 10: Final sanity check – confirm the VPC is gone.
# If it still exists, print a warning with what's blocking it.
- name: Verify Cleanup
  if: always()
  run: |
    VPC_ID=$(aws ec2 describe-vpcs \
        --filters "Name=tag:Name,Values=shipzen-vpc" \
        --query "Vpcs[0].VpcId" \

```



```

--output text 2>/dev/null || echo "")

if [ -z "$VPC_ID" ] || [ "$VPC_ID" = "None" ]; then
    echo "✅ VPC fully deleted. All infrastructure is down."
else
    echo "⚠️ VPC $VPC_ID still exists. Listing remaining dependencies:"
    echo "--- Network Interfaces ---"
    aws ec2 describe-network-interfaces \
        --filters "Name=vpc-id,Values=$VPC_ID" \
        --query "NetworkInterfaces[*].{ID:NetworkInterfaceId,Desc:Description,Statu
        --output table || true
    echo "--- Security Groups ---"
    aws ec2 describe-security-groups \
        --filters "Name=vpc-id,Values=$VPC_ID" \
        --query "SecurityGroups[?GroupName!='default'].{ID:GroupId,Name:GroupName}"
        --output table || true
fi

# Step 11: Ultimate Nuclear Cleanup
# Kubernetes controllers (like EBS CSI) and dynamic scripts often leave behind
# resources that Terraform doesn't track in its state file.
# This step aggressively sweeps the account for anything left behind.
- name: Nuke Orphaned Volumes, Buckets, and IPs
  if: always()
  run: |
    echo "🧹 Sweeping for orphaned EBS Volumes (dynamically provisioned by Kubernet
    VOLUMES=$(aws ec2 describe-volumes \
        --query "Volumes[?Tags != null] | [?contains(to_string(Tags), 'shipzen')].Vol
        --output text 2>/dev/null || echo "")

    for VOL in $VOLUMES; do
        echo "Deleting orphaned EBS Volume: $VOL"
        aws ec2 delete-volume --volume-id "$VOL" || true
    done

    echo "🧹 Sweeping for orphaned S3 Buckets..."
    BUCKETS=$(aws s3api list-buckets --query "Buckets[?contains(Name, 'shipzen')].N

    for BUCKET in $BUCKETS; do
        echo "Emptying and deleting orphaned S3 Bucket: $BUCKET"
        aws s3 rm "s3://$BUCKET" --recursive || true
        # Sometimes delete fails if there are lingering delete markers or versions
        aws s3api delete-bucket --bucket "$BUCKET" || true
    done

```

```
echo " 🚧 Sweeping for unassociated Elastic IPs..."
# Find any EIPs that are not associated with an instance/ENI and are tagged with
EIPS=$(aws ec2 describe-addresses \
  --query "Addresses[?AssociationId==null && Tags[?contains(Value, 'shipzen')]]"
  --output text 2>/dev/null || echo "")

for EIP in $EIPS; do
  echo "Releasing orphaned Elastic IP: $EIP"
  aws ec2 release-address --allocation-id "$EIP" || true
done

echo " ✅ Nuclear sweep complete. Account is clean."
```

File: `.github\workflows\security-scan.yaml`

Purpose & Responsibility: This file is a critical piece of the `C:\Project\ShipZen\.github` subsystem. It manages the core logic for its respective domain.

Interview Question: *Why did you structure this file this way?* **Answer:** *To maintain separation of concerns and ensure it can be independently tested and scaled. Removing this file would cause a catastrophic failure in the event-driven lifecycle.*

Source Code:

```
name: Container Security Scan (Trivy)

on:
  schedule:
    - cron: '0 2 * * *' # Run nightly at 2 AM
  workflow_dispatch:

permissions:
  contents: read

jobs:
  trivy-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Run Trivy vulnerability scanner on local code
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          ignore-unfixed: true
          format: 'table'
          severity: 'CRITICAL,HIGH'
```

8. Failure Scenarios & Troubleshooting Runbook

Incident 1: HTTPRoute Missing

- **Symptoms:** Users report 404 from Envoy.
- **Root Cause:** Controller validation error skipped route creation.
- **Resolution:** Verified controller logs and updated Jinja template. Ensure loop checks all resources.

9. Interview Cheat Sheet

- **Karpenter:** JIT Node provisioning.
- **Envoy:** Gateway API routing.
- **Kaniko:** Daemonless image building.
- **PostgreSQL:** Single source of truth.